HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

HỌC PHẦN: LẬP TRÌNH MẠNG

BÁO CÁO BÀI TẬP LỚN

JUMPKING MULTIPLAYER

Thiết kế và đánh giá kiến trúc
đồng bộ trạng thái thời gian thực
và điều phối thực thi theo phòng
cho trò chơi trực tuyến nhiều người chơi

Giảng viên hướng dẫn: **Ngô Văn Quyền**

Nhóm lớp: .....

Nhóm BTL: .....

Thành viên thực hiện

STT	Mã sinh viên	Họ và tên
1	B23DCCN067	Nguyễn Văn Bằng
2	B23DCCN863	Tô Quang Trung
3	B23DCCN835	Phạm Quang Toản
4	B23DCCN809	Lê Thanh Thuỷ
5	B23DCCN686	Đặng Xuân Quang

Hà Nội – 2026

Nhận xét của giảng viên chấm bài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên *(ký và ghi rõ họ tên)*: .....

Tóm tắt

Trò chơi trực tuyến đòi hỏi sự phối hợp giữa phản hồi tương tác, quyền quyết định trạng thái và độ bền dữ liệu. Đối với thao tác nhảy tích lực, độ trễ và sai lệch thứ tự input có thể làm thay đổi quỹ đạo quan sát được, trong khi kết quả thi đấu vẫn cần được xác lập độc lập với client. Nghiên cứu thiết kế và hiện thực kiến trúc có thẩm quyền máy chủ cho JumpKing Multiplayer, kết hợp mô phỏng Java ở 60 Hz, dự đoán và hiệu chỉnh TypeScript, cùng luồng snapshot WebSocket ở 20 Hz. Trạng thái được xử lý tuần tự theo phòng trên worker pool dùng chung; truyền mạng và lưu trữ được tách khỏi vòng mô phỏng. Giao thức sử dụng sequence và generation để xác định ngữ cảnh điều khiển, trong khi revision và giao dịch bảo vệ checkpoint cùng kết quả đã lưu.

Trong mô hình có độ trễ 100 ms mỗi chiều, trung vị thời gian quan sát cú nhảy giảm từ 566,7 ms xuống 350,0 ms, tương ứng 216,7 ms (38,2%), với cùng thời gian tích lực. Năm lần chạy lại xác nhận tính tái lập của kết quả trong mô hình xác định. Kiểm chứng chức năng đạt 176 kiểm thử phía client, 118 kiểm thử đơn vị Java và 16 kiểm thử tích hợp chọn lọc trên MySQL cô lập. Phép đo tải tham chiếu hỗ trợ phân tích chi phí truyền trạng thái; phạm vi suy luận được giới hạn bởi môi trường loopback và thời lượng lấy mẫu. Đóng góp của nghiên cứu là một thiết kế tích hợp bảo toàn ngữ nghĩa điều khiển, quyền sở hữu trạng thái và tính nhất quán giao dịch, cùng phương pháp kiểm chứng theo từng phạm vi bảo đảm.

Từ khóa: server-authoritative; client prediction; reconciliation; WebSocket; concurrency; checkpoint consistency.

Mục lục

Tóm tắt **i**

Danh mục thuật ngữ và chữ viết tắt **iv**

1 Giới thiệu **1**
1.1 Bối cảnh và đặc trưng bài toán 1
1.2 Vấn đề nghiên cứu và các thuộc tính cần bảo toàn 1
1.3 Mục tiêu và câu hỏi nghiên cứu 2
1.4 Phương pháp nghiên cứu và phạm vi kiểm chứng 3
1.5 Phạm vi và đóng góp 3

2 Cơ sở lý thuyết và công trình liên quan **4**
2.1 Kiến trúc game mạng và ranh giới tin cậy 4
2.2 Các công trình liên quan và giả định áp dụng 5
2.3 Mô phỏng, dự đoán và tính tương đương 6
2.4 Các mô hình thực thi và quản lý dữ liệu 6
2.5 Tổng hợp lựa chọn và khoảng trống kỹ thuật 8

3 Phát biểu bài toán **10**
3.1 Yêu cầu và phạm vi phục vụ 10
3.2 Mô hình trạng thái, thành phần và định danh 11
3.3 Mô hình lỗi và điều kiện khôi phục 12
3.4 Các bất biến và tiêu chí kiểm chứng 12

4 Phương pháp, kiến trúc và giao thức **14**
4.1 Nguyên lý và kiến trúc đồng bộ 14
4.2 Giao thức và vòng đời thông điệp 15
4.3 Thuật toán tiếp nhận và hiệu chỉnh input 17
4.4 Tương đương mô phỏng và trình bày trạng thái 19
4.5 Đánh đổi thiết kế và trình tự xác nhận 19

5 Hiện thực và độ tin cậy **21**
5.1 Ownership và chính sách thực thi theo phòng 21
5.2 Ngân sách tick và truyền trạng thái 22

5.3 Giao dịch và tính nhất quán của persistence 23
5.4 Vòng đời và xử lý tình huống cạnh tranh 25
5.5 Bảo mật và mô hình triển khai 26

6 Thực nghiệm và thảo luận 28
6.1 Thiết kế thực nghiệm và metric 28
6.2 Kết quả đối chứng prediction 30
6.3 Kiểm chứng chức năng và tính nhất quán 31
6.4 Tải tham chiếu và phạm vi suy luận 32
6.5 Thảo luận kết quả và ưu tiên cải tiến 33

7 Đóng góp và kết luận 35
7.1 Đóng góp và bằng chứng 35
7.2 Trả lời câu hỏi nghiên cứu và ý nghĩa kết quả 36
7.3 Giới hạn của nghiên cứu 37
7.4 Hướng phát triển 37
7.5 Kết luận 38

A Điều kiện và khả năng tái lập thực nghiệm 41
A.1 Phiên bản và môi trường 41
A.2 Tham số đo và hồ sơ đối chứng 41

Danh mục thuật ngữ và chữ viết tắt

Thuật ngữ	Ý nghĩa trong nghiên cứu
ACK	Acknowledgment; xác nhận input đã được máy chủ tiêu thụ trong mô phỏng.
Checkpoint	Phần trạng thái tiến độ được ghi bền vững để khôi phục lượt Community.
CSRF	Cross-Site Request Forgery; giả mạo yêu cầu từ một ngữ cảnh web khác.
FIFO	First In, First Out; thứ tự vào trước, ra trước của một hàng đợi.
Generation	Thế hệ ngữ cảnh điều khiển; phân biệt lịch sử input trước và sau khi ngữ cảnh được thiết lập lại.
HTTP	Hypertext Transfer Protocol; giao thức cho xác thực, truy vấn và thao tác tài nguyên.
Interpolation	Nội suy trạng thái hiển thị giữa các snapshot đã nhận của người chơi khác.
Ownership	Quyền sở hữu trạng thái, xác định thành phần được phép quyết định cập nhật.
p50, p95	Phân vị 50% và 95% của tập mẫu, dùng để mô tả phần giữa và phần đuôi phân bố.
Prediction	Dự đoán phía client bằng mô phỏng input cục bộ trước khi nhận xác nhận từ server.
Reconciliation	Hiệu chỉnh từ snapshot có thẩm quyền rồi phát lại phần input chưa được xác nhận.
Revision	Phiên bản dữ liệu dùng để kiểm soát thứ tự chấp nhận cập nhật khi ghi.
RQ	Research Question; câu hỏi nghiên cứu định hướng thiết kế và đánh giá.
RTT	Round-Trip Time; thời gian trao đổi hai chiều, không đồng nhất với độ trễ một chiều.
WebSocket / WSS	Kết nối truyền thông hai chiều; WSS sử dụng lớp bảo vệ TLS.

Danh sách hình vẽ

4.1 Ba mốc xác nhận và ba ý nghĩa khác nhau của trạng thái. 14
4.2 Kiến trúc logic và vị trí các đóng góp C1–C4; mũi tên biểu diễn trách nhiệm trao đổi. 15
4.3 Diễn tiến hiệu chỉnh trong cùng một generation và cách loại lịch sử khi đổi ngữ cảnh. Sequence là giá trị minh họa, không phải số đo thực nghiệm; khoảng cách trên hình không biểu diễn độ trễ. 17
4.4 Luồng tổng hợp: ACK gameplay và xác nhận commit có ngữ nghĩa khác nhau. . 20

5.1 Quan hệ dữ liệu rút gọn từ schema: lượt Community không liên kết match; lượt Room tham chiếu match. 24
5.2 Các trạng thái Room chính; reconnect không tự tạo trận mới. 27

6.1 Biểu đồ dựng lại từ bảng lịch sử; không phải phép đo mới. 30
6.2 Sai lệch so với mô phỏng tham chiếu áp input ngay trong mô hình xác định E_P; không phải khoảng cách tới server tại cùng thời gian thực. 32

Danh sách bảng

1.1 Các thuộc tính cần quan sát ở những tầng khác nhau. 2

2.1 Các phương án liên quan và vị trí của dự án. 6
2.2 Các câu hỏi dùng để phân tích một phương án đồng bộ. 8
2.3 So sánh công trình theo bài toán, đồng bộ, concurrency và giới hạn áp dụng. . . 8

3.1 Phân biệt các tình trạng thường cùng xuất hiện dưới thông báo mất kết nối. . . . 12
3.2 Từ bất biến đến tình huống kiểm tra. 13

4.1 Các loại định danh không thể thay thế cho nhau. 16
4.2 Các phương án thiết kế được đối chiếu theo ngữ nghĩa và chi phí. 20

5.1 Một interleaving hợp lệ của hai loại queue. 22
5.2 Chính sách xử lý theo loại dữ liệu bất đồng bộ. 23

6.1 Metric, phạm vi và diễn giải cần tránh. 29
6.2 So sánh trong mô hình có kiểm soát [13]. 30
6.3 Kết quả mô hình qua năm lượt: trung vị trùng khớp, đơn vị ms. 31
6.4 Số liệu tải tham chiếu E_H; môi trường loopback [13]. 32
6.5 Thời gian compute tick lịch sử, đơn vị ms [13]. 33

7.1 Cơ chế, giá trị kỹ thuật và bằng chứng của bốn nhóm đóng góp. 36
7.2 Tổng hợp câu trả lời theo loại bằng chứng. 36

A.1 Môi trường của các tập bằng chứng. 41

Chương 1

Giới thiệu

1.1 Bối cảnh và đặc trưng bài toán

Jump King là trò chơi mà người chơi phải điều chỉnh hướng và thời gian tích lực để vượt địa hình. Một thao tác sai có thể đánh đổi bằng việc mất phần lớn độ cao đã đạt được. Vì vậy, cảm nhận độ trễ không chỉ là vấn đề thẩm mỹ: nó ảnh hưởng đến khả năng người chơi thiết lập quan hệ giữa thao tác, quỹ đạo và hậu quả. Khi đưa dạng gameplay này lên môi trường nhiều người, hệ thống phải đồng thời đáp ứng tính liên tục của điều khiển và tính đáng tin cậy của kết quả.

Một client chỉ hiển thị trạng thái nhận từ server sẽ dễ triển khai quyền quyết định tập trung, nhưng phản hồi thị giác phải chờ trao đổi qua mạng. Ngược lại, giao hoàn toàn vị trí và thời gian cho client làm giảm độ trễ cảm nhận nhưng mở rộng ranh giới tin cậy tới thiết bị do người chơi kiểm soát. Bài toán thiết kế là tổ chức hai vai trò này sao cho phản hồi nhanh không đồng nghĩa với trao quyền xác lập kết quả.

Một lượt chơi trực tuyến tạo ra nhiều dòng sự kiện có quan hệ nhân quả nhưng không hoàn tất đồng thời. Trình duyệt thu nhận thao tác, mạng chuyển thông điệp, máy chủ thực hiện bước mô phỏng và database xác nhận giao dịch theo những lịch độc lập. Sự khác biệt giữa các lịch này là nguồn gốc của những tình huống mà quan sát cục bộ và kết quả chính thức chưa trùng khớp. Do đó, đơn vị phân tích của nghiên cứu là quá trình chuyển trạng thái qua các thành phần, không phải từng màn hình giao diện.

Ví dụ, người chơi có thể giữ Jump rồi mở menu trước khi máy chủ xử lý toàn bộ input. Một thiết kế xem trạng thái không nhận thêm input như một lần thả phím có thể tạo ra cú nhảy ngoài ý muốn. Trong trường hợp khác, client đã gửi yêu cầu lưu nhưng mất kết nối trước khi nhận phản hồi. Việc phát lại thao tác mà không xác định định danh lượt có thể tác động tới một ngữ cảnh mới. Hai ví dụ cho thấy tính đúng của hệ thống phụ thuộc cả nội dung thông điệp lẫn thứ tự và ngữ cảnh áp dụng.

1.2 Vấn đề nghiên cứu và các thuộc tính cần bảo toàn

Dự án giải quyết câu hỏi: làm thế nào xây dựng một game nhiều người trên trình duyệt, trong đó client phản hồi sớm theo input, server vẫn quyết định trạng thái và dữ liệu lưu dài hạn không

mâu thuẫn với kết quả đã xác nhận? Vấn đề bao gồm cả những trường hợp không thuận lợi: input đến dồn, socket bị đóng, người chơi mở nhiều tab, ghi database thất bại và server khởi động lại.

Tính đúng đắn vì thế có ít nhất ba mốc. Nhân vật phản ứng trên màn hình chưa đồng nghĩa server đã xử lý input. Server xử lý input chưa đồng nghĩa checkpoint đã được commit. Ba mốc xác nhận tương ứng với ba phạm vi nhất quán: hiển thị cục bộ, mô phỏng có thẩm quyền và dữ liệu bền vững.

Tính đáp ứng mô tả khoảng thời gian từ thao tác đến phản hồi quan sát được. Trong game tích lực, thời gian này chứa cả thành phần có chủ đích của luật chơi và thành phần chờ truyền thông. Một phép so sánh hợp lệ phải giữ thời gian charge như nhau, để không nhầm việc thay đổi luật với việc giảm độ trễ mạng cảm nhận. Prediction được đánh giá theo vai trò giảm phần chờ trong đường hiển thị.

Một hành động chỉ có giá trị chính thức khi nó được xử lý trong đúng ngữ cảnh và theo luật máy chủ. Tính đúng được quan sát qua input đã consume, trạng thái mô phỏng, quyền thao tác và kết quả vòng đời. Hình ảnh mượt không đủ xác nhận thuộc tính này, vì renderer có thể đang hiển thị một dự đoán chưa được máy chủ chấp nhận. Vì vậy, kiểm thử cần oracle nằm ngoài giao diện.

Database lưu một phần trạng thái theo chính sách nghiệp vụ. Checkpoint được commit có thể cũ hơn trạng thái đang hiển thị; kết quả đã xác lập trong RAM có thể đang chờ ghi. Nghiên cứu phân tích khoảng cách đó qua revision, hàng đợi writer và các mốc ghi bắt buộc; Bảng 1.1 phân định oracle theo từng tầng. Độ bền không được suy từ việc WebSocket vẫn mở hoặc nhân vật đã chạm đích.

Bảng 1.1: Các thuộc tính cần quan sát ở những tầng khác nhau.

Thuộc tính	Đại lượng hoặc oracle	Thành phần quyết định
Phản hồi	Thời gian quan sát hành động	Client prediction và presentation
Đúng luật	State và ACK sau consume	Simulation trên server
Đúng quyền	Owner, Host, ngữ cảnh hợp lệ	Session và lớp nghiệp vụ
Bền vững	Revision và kết quả commit	Service transaction và database

1.3 Mục tiêu và câu hỏi nghiên cứu

1. RQ1: Server-authoritative và prediction/reconciliation đáp ứng tính tương tác và nhất quán như thế nào?
2. RQ2: Xử lý tuần tự theo phòng trên shared worker pool ảnh hưởng tới correctness và performance ra sao?
3. RQ3: Tiến độ và kết quả được bảo vệ thế nào khi reconnect, duplicate requests hoặc database lỗi?
4. RQ4: Những đánh đổi về latency, tài nguyên, reliability và scalability là gì?

Mục tiêu là trả lời từng câu hỏi bằng cơ chế cụ thể và căn cứ phù hợp. RQ1 được đánh giá bằng đối chứng mô hình và các lượt tái thực hiện; RQ2–RQ3 chủ yếu được phân tích bằng implementation và kiểm thử tình huống; RQ4 tổng hợp các đánh đổi trong phạm vi những bằng chứng này.

1.4 Phương pháp nghiên cứu và phạm vi kiểm chứng

Nghiên cứu kết hợp phân tích thiết kế với kiểm chứng thực thi. Phân tích thiết kế xác định state, ownership và các bất biến. Truy vết mã nguồn kiểm tra xem những điều kiện đó được thực hiện ở đâu. Kiểm thử tình huống tạo ra các thứ tự sự kiện có rủi ro. Cuối cùng, đối chứng mô hình đánh giá một đại lượng cụ thể trong điều kiện được kiểm soát.

Các bước này tạo thành chuỗi giải thích: một yêu cầu được chuyển thành cơ chế, cơ chế sinh ra thuộc tính cần kiểm tra, và kết quả kiểm tra giới hạn phát biểu kết luận. Cách tiếp cận giới hạn kết luận theo các cơ chế và điều kiện được kiểm chứng, phù hợp với nghiên cứu thiết kế hệ thống ở quy mô phòng nhỏ.

Một hệ thống chạy được qua mạng là điều kiện cần để đánh giá nhưng chưa xác định được nguyên nhân của chất lượng quan sát. Hình ảnh phản hồi nhanh có thể xuất phát từ prediction, nhịp renderer, thời gian charge khác nhau hoặc một đường truyền thuận lợi. Nghiên cứu vì vậy xác định biến cần khảo sát trước khi so sánh. Với RQ1, biến đó là chiến lược đồng bộ local player; các điều kiện luật chơi được giữ cố định trong mô hình.

Đối với concurrency và persistence, một benchmark tổng hợp không thể tự tách đóng góp của từng cơ chế. Nếu thời gian phản hồi tăng, nguyên nhân có thể nằm ở queue, scheduling, serialization hoặc database. Bởi vậy, nghiên cứu kết hợp hai kiểu đánh giá: kiểm tra bất biến để xác nhận một cơ chế đang thực hiện đúng nhiệm vụ và đo có đối chứng để định lượng một ảnh hưởng được cô lập. Sự phân công này tránh dùng số PASS như bằng chứng tăng throughput, hoặc dùng RTT thấp như bằng chứng transaction đã nhất quán.

Các RQ định hướng việc lựa chọn metric và oracle trước khi thực nghiệm. Phân tích cơ chế được đối chiếu với mã nguồn; kết quả đo được diễn giải theo cùng ranh giới trạng thái và điều kiện lỗi đã xác lập.

Khả năng tái lập được hiểu ở hai mức. Mức cấu trúc cho phép truy từ luận điểm tới source và test. Mức thực thi cho phép chạy cùng trace hoặc workload bằng lệnh và môi trường đã công bố. Việc giữ dữ liệu gốc, thông tin phiên bản và kết quả không hoàn tất giúp người đọc phân biệt một phép đo có thể sử dụng với một lần chạy chưa tạo đủ bằng chứng. Đây là cơ sở để các vòng phát triển sau có thể so sánh với nghiên cứu hiện tại theo cùng tiêu chí.

1.5 Phạm vi và đóng góp

Phạm vi gồm Community lưu tiến độ cá nhân, Room thi đấu nhiều người, xác thực, chat, thành tích và khôi phục kết nối trên một backend. Bốn đóng góp là tích hợp mô phỏng Java/TypeScript (C1), bảo toàn ngữ nghĩa input (C2), xử lý phòng với I/O tách biệt (C3) và persistence nhất quán (C4). Chương 7 phân tích cơ chế, phần kế thừa và bằng chứng của từng đóng góp.

Chương 2

Cơ sở lý thuyết và công trình liên quan

2.1 Kiến trúc game mạng và ranh giới tin cậy

Trong mô hình có thẩm quyền, client gửi ý định điều khiển thay vì kết quả cần được tin cậy. Server áp dụng luật để xác lập vị trí, thời gian và trạng thái kết thúc. Lựa chọn này thu hẹp khả năng client tự khai chiến thắng, nhưng không tự loại bỏ bot, phá hoại tài nguyên hoặc mọi hình thức gian lận. Chính sách quyền, giới hạn tài nguyên và kiểm tra input vẫn cần được thiết kế riêng.

Một phương án khác là client-authoritative, phù hợp khi ưu tiên trải nghiệm cục bộ nhưng làm yếu cơ sở xác thực kết quả chung. Lockstep đồng bộ input giữa các bên có thể giảm truyền trạng thái, song yêu cầu tính xác định và chính sách chờ/đồng bộ phức tạp. Dự án chọn server-authoritative phù hợp với leaderboard và kết quả trận có người quản lý tập trung.

RFC 6455 định nghĩa giao tiếp WebSocket hai chiều [1]. Giao thức truyền không quyết định việc input đã được mô phỏng hay dữ liệu đã được lưu. Bởi vậy, server vẫn cần ACK ở tầng game và thông báo lưu sau giao dịch. Việc chọn JSON giúp kiểm tra và debug dễ hơn, đánh đổi bằng kích thước và chi phí tuần tự hóa so với một biểu diễn nhị phân được thiết kế tương đương.

Kiến trúc game mạng trước hết là một lựa chọn về quyền cập nhật state. Nếu mỗi client được quyền công bố tọa độ chính thức, server chủ yếu điều phối thông tin nhưng phải xử lý độ tin cậy của giá trị nhận được. Nếu server giữ toàn bộ quyền quyết định, client gửi ý định và chấp nhận correction. Phân tán quyền theo đối tượng hoặc vùng tiếp tục đặt ra bài toán ai là chủ khi đối tượng chuyển vùng và các thao tác liên vùng được sắp thứ tự thế nào.

Đối với JumpKing, trạng thái cần xác nhận gồm quỹ đạo, clock, tiến độ và điều kiện kết thúc. Quyền cập nhật tập trung cho phép các kết quả được tính từ cùng một luật thay vì từ các phép khai báo của người chơi. Cái giá là server trở thành điểm tập trung công việc và nguồn trạng thái bắt buộc khi cần hiệu chỉnh. Đây là đánh đổi thiết kế, không phải đặc tính được giải quyết chỉ bằng lựa chọn framework.

Prediction không loại bỏ thời gian truyền thông. Nó thay đổi đường đi tạo phản hồi thị giác: client tự mô phỏng một kết quả tạm thời trong lúc server xử lý. Interpolation của người chơi khác thực hiện lựa chọn khác: trì hoãn điểm hiển thị để có các mẫu hai bên. Hai cơ chế vì thế phân bổ độ trễ theo vai trò local player và remote player, thay vì tối ưu cùng một metric.

Trong trò chơi không có va chạm trực tiếp giữa mọi người chơi, chính sách hiển thị peers có thể được tách tương đối khỏi thao tác điều khiển bản thân. Khi có tác động Items do server quyết định, client vẫn phải tiếp nhận trạng thái chính thức của sự kiện đó. Phạm vi dự đoán được xác định theo khả năng tái tạo luật và dữ liệu sẵn có tại client, không theo giả định rằng mọi tương tác đều có thể suy đoán chính xác.

2.2 Các công trình liên quan và giả định áp dụng

Tổng quan này là một khảo sát có định hướng theo cơ chế, không tự nhận là systematic review bao phủ toàn bộ tài liệu. Các nguồn được chọn phải liên hệ trực tiếp với ít nhất một RQ, có tác giả hoặc tổ chức công bố xác định và có nội dung kiểm tra được. Bài nghiên cứu, technical report, chuẩn giao thức và tài liệu thực hành được phân biệt về loại bằng chứng. Không so sánh số throughput giữa các paper khi workload và phần cứng khác nhau.

Colyseus chia đối tượng game giữa các node, duy trì primary cho cập nhật và các replica để đọc, kết hợp tìm kiếm đối tượng với prefetch theo mức liên quan không gian [2]. Giá trị so sánh đối với JumpKing nằm ở ranh giới ownership: một bên phân tán ownership theo đối tượng trên nhiều máy, bên còn lại giữ phòng trong một backend. Cả hai đều phải phân biệt nơi quyết định cập nhật với nơi đọc trạng thái trễ. Tuy nhiên, quyền điều khiển Room trong JumpKing không phải cơ chế placement hoặc failover như một kiến trúc phân tán. Không thể lấy kết quả scale của Colyseus làm bằng chứng cho sức chứa JumpKing. Đây cũng là lý do hướng phát triển nhiều backend cần giải quyết ownership và routing trước khi thêm load balancer.

Mauve và cộng sự nghiên cứu trạng thái thay đổi theo cả hành động lẫn thời gian, kết hợp local-lag và Timewarp để xử lý nhất quán trong ứng dụng liên tục được nhân bản [3]. Điểm liên quan là không thể mô hình game chỉ như một chuỗi request CRUD: thời gian giữa các thao tác cũng tác động state. JumpKing chọn phản hồi dự đoán rồi sửa theo server, thay vì cố ý trì hoãn mọi hành động cục bộ để chờ nhất quán. Reconciliation có thao tác replay nhưng không đồng nghĩa implementation đã triển khai đầy đủ Timewarp của công trình. Sự khác biệt nằm ở nguồn quyền quyết định và phạm vi lịch sử được giữ để sửa trạng thái.

Orleans nâng mô hình actor lên virtual actors và cung cấp hỗ trợ runtime cho các vấn đề phân tán [4]. JumpKing chỉ kế thừa ý tưởng xử lý tuần tự trên một chủ thể state: SerialExecutor không cung cấp tự động chuyển actor qua máy khác, định vị toàn cluster hoặc phục hồi activation. Chính giới hạn này giúp mô hình học phần dễ vận hành hơn, đồng thời giới hạn fault tolerance ở cấp process.

SEDA tổ chức dịch vụ thành các stage có queue, gắn với điều khiển tài nguyên dưới tải [5]. Send worker và persistence worker của JumpKing có nét tương đồng về tách công đoạn, nhưng bounded queue hiện tại không tự cấu thành bộ điều khiển adaptive như SEDA. Cần đo queue age và deadline để biết hệ thống đang hoạt động trong vùng chịu tải hay chỉ từ chối thêm việc. Coalescing snapshot có thể bỏ thông tin cũ vì bản mới thay thế được nó; không được suy rộng chính sách đó sang control message hoặc cạnh nhấn/thả Jump.

Claypool và Claypool phân tích hành động người chơi theo yêu cầu độ chính xác và thời hạn phản ứng [6]. Từ góc nhìn đó, JumpKing có lý do ưu tiên phản hồi input và bảo toàn charge: cùng một độ trễ có thể gây hậu quả khác nhau giữa chat và điều khiển cú nhảy. Tuy nhiên, báo

cáo chưa có thí nghiệm người dùng để đo hiệu quả hành vi hoặc cảm nhận. Kết quả latency trong harness chỉ là bằng chứng kỹ thuật, chưa đủ kết luận người chơi hoàn thành game tốt hơn.

2.3 Mô phỏng, dự đoán và tính tương đương

Tài liệu của Gambetta mô tả cách đánh số input để client biết phần nào đã được server xử lý; sau khi nhận trạng thái chính thức, client bỏ input đã ACK và phát lại phần còn lại [7]. Dự án kế thừa nguyên lý này, bổ sung quản lý ngữ cảnh lượt chơi và cơ chế hủy input phù hợp với thao tác tích lực. Đây là đóng góp tích hợp theo miền bài toán, không phải sáng tạo nguyên lý prediction.

Fixed timestep giúp tách bước vật lý khỏi tốc độ dựng hình. Fiedler phân tích cách tổ chức bước thời gian và nguy cơ xử lý bù vượt khả năng tính toán [8]. Dự án chọn bước mô phỏng cố định; interpolation của peers là cơ chế trình bày giữa các snapshot, không được dùng để xác lập kết quả. Vì vậy, mượt hình và đúng luật là hai mục tiêu liên quan nhưng không đồng nhất.

Java và TypeScript sử dụng các môi trường số học khác nhau. Việc đọc cùng JSON chỉ đồng nhất tham số; nó không đồng nhất thứ tự tính toán, phép làm tròn hoặc cách kiểm tra biên. Với một game có nhiều bước cập nhật liên tiếp, sai khác nhỏ ở một phép so sánh collision có thể thay đổi kết quả grounded, rồi thay đổi khả năng launch ở các tick sau.

Vì vậy, tính tương đương được tiếp cận theo các lớp. Lớp dữ liệu kiểm tra map và tuning. Lớp chuyển trạng thái kiểm tra fixtures có expected state. Lớp chuỗi hành vi kiểm tra replay qua nhiều tick và các mốc địa hình. Mỗi lớp loại trừ một nhóm sai khác nhưng vẫn hữu hạn; giá trị của chúng nằm ở khả năng phát hiện regression có ý nghĩa đối với luật chơi. Đây là một nghĩa thực dụng của kiểm chứng tương đương, khác với chứng minh toán học trên mọi input.

2.4 Các mô hình thực thi và quản lý dữ liệu

Locking reads của InnoDB cung cấp công cụ phối hợp cập nhật cạnh tranh [9]. Tuy nhiên, có database hỗ trợ transaction không đồng nghĩa mọi thao tác game tự nhất quán: nhóm phải chọn đúng đối tượng khóa, phạm vi giao dịch và điều kiện chấp nhận revision. Tương tự, worker pool cung cấp khả năng thực thi song song nhưng không tự xác lập ai sở hữu state.

Bảng 2.1: Các phương án liên quan và vị trí của dự án.

Phương án	Điểm mạnh	Giới hạn đối với bài toán
Snapshot-only	Trình bày gần trạng thái server	Phản hồi phải chờ mạng và nhịp snapshot
Client-authoritative	Phản hồi cục bộ trực tiếp	Không đủ cơ sở tin cậy kết quả client khai
Prediction và reconciliation	Phản hồi sớm, có sửa theo server	Cần replay, quản lý input và state tương đương
Ghi DB trong tick	Luồng điều khiển dễ theo dõi	I/O có thể ảnh hưởng nhịp mô phỏng
Worker persistence	Tách chờ I/O khỏi tick	Phải giải quyết thứ tự, retry và commit

Các phương án ở Bảng 2.1 khác nhau trước hết ở ranh giới ownership. Một chương trình có nhiều thread chưa nhất thiết là một hệ thống có ownership phân tán. Trong dự án, worker pool

chia sẻ tài nguyên CPU của một process, còn executor phòng phân định thứ tự thay đổi state. Hai tầng này giải quyết những câu hỏi khác nhau: tác vụ chạy bằng tài nguyên nào và tác vụ được phép quan sát/cập nhật state theo thứ tự nào.

Sự phân biệt đặc biệt quan trọng khi đối chiếu với Orleans hoặc Colyseus. Tổ chức task tuần tự trong một process không tự cung cấp tìm kiếm chủ thể qua mạng, di chuyển ownership, thay thế node chết hoặc bộ nhớ bền vững. Những khả năng đó cần cơ chế bổ sung và có thể làm thay đổi cả ngữ nghĩa giao thức. Vì vậy, nghiên cứu sử dụng các công trình phân tán như phương án đối chiếu kiến trúc, không coi JumpKing đã có tính năng của chúng chỉ vì có khái niệm phòng hoặc actor-like execution.

Một thiết kế cho nhiều luồng cập nhật trực tiếp state có thể dùng khóa để bảo vệ từng đối tượng hoặc thao tác. Để đúng, phạm vi khóa phải bao phủ những trường cần quan sát nhất quán; thứ tự lấy nhiều khóa cũng cần được kiểm soát. Với một chuyển trạng thái gồm physics, clock và kết quả, khóa quá nhỏ có thể để lọt trạng thái trung gian, còn khóa quá lớn có thể kéo dài thời gian chờ.

Ownership theo phòng chọn một hướng khác: các cập nhật gameplay đi qua cùng chuỗi thực thi. Nó làm giảm nhu cầu phối hợp khóa bên trong state phòng, nhưng tăng yêu cầu mọi đường cập nhật tuân thủ executor. Khi một callback thoát khỏi quy tắc này, lợi ích suy luận tuần tự không còn áp dụng cho toàn phòng. Do đó, việc chọn actor-like execution chuyển trọng tâm kiểm chứng từ từng lock sang ranh giới sở hữu và đường callback; nó không loại bỏ hoàn toàn vấn đề concurrency.

Thread riêng cung cấp một đường thực thi dễ nhận diện nhưng số lượng thread tăng theo số phòng. Worker pool dùng chung tách số phòng logic khỏi số tài nguyên thực thi; phòng không hoạt động không nhất thiết giữ một thread. Đổi lại, các phòng có thể cạnh tranh cùng pool, và task dài của một phòng có thể ảnh hưởng scheduling của phòng khác.

Với mục tiêu nhiều phòng nhỏ trong một backend, lựa chọn worker pool có cơ sở về tổ chức tài nguyên. Tuy nhiên, mức hiệu quả cụ thể phụ thuộc số worker, độ dài task, chính sách drain và workload. Báo cáo do đó phân tích tính phù hợp kiến trúc và cung cấp kiểm chứng chức năng, còn mức tăng hiệu năng so với thread-per-room là một câu hỏi cần ablation riêng.

Ghi database ngay trong đường xử lý input tạo một quan hệ điều khiển trực tiếp: thao tác chỉ tiếp tục sau khi việc ghi kết thúc. Cách này đơn giản hóa một số luồng xác nhận nhưng đưa thời gian JDBC, khóa và commit vào đường đáp ứng. Với tick định kỳ, một lần I/O dài có thể ảnh hưởng nhiều người cùng phòng nếu nằm trong task sở hữu state.

Persistence bất đồng bộ đưa việc chờ ra worker, nhưng tạo thêm khoảng cách giữa state RAM và dữ liệu bền vững. Revision, payload bất biến và barrier là chi phí thiết kế để quản lý khoảng cách đó. Việc đánh giá lựa chọn cần nhìn cả hai mặt: bảo vệ nhịp mô phỏng và giải thích chính xác thời điểm được thông báo đã lưu. Bỏ qua mặt thứ hai có thể tạo trải nghiệm nhanh nhưng sai về xác nhận dữ liệu.

2.5 Tổng hợp lựa chọn và khoảng trống kỹ thuật

Bảng 2.2: Các câu hỏi dùng để phân tích một phương án đồng bộ.

Trục phân tích	Câu hỏi thiết kế	Áp dụng cho dự án
Quyền quyết định	Ai được xác nhận cập nhật?	Server xác lập physics và kết quả.
Thứ tự	Message đến và hành động có cùng thứ tự không?	Chỉ ACK sau consume; phân biệt queue với simulation.
Phản hồi	Hình ảnh nào xuất hiện trước xác nhận?	Local prediction; remote interpolation.
Ownership	Thành phần nào được sửa state?	Executor sở hữu phòng.
Quá tải	Khi chậm, dữ liệu nào được thay thế?	Snapshot có thể coalesce; cạnh input cần được bảo toàn.
Lỗi	Điều gì còn lại sau disconnect/restart?	Membership, checkpoint và Room có vòng đời khác nhau.
Lưu trữ	Khi nào được gọi là đã lưu?	Sau giao dịch thành công.

Khung đối chiếu ở Bảng 2.2 đặt các quyết định vào cùng hệ quy chiếu về quyền, thứ tự, phục hồi và lưu trữ. Nó cũng giúp nhận diện những phép so sánh chưa thể thực hiện: một giải pháp dùng định dạng nhị phân không tự tốt hơn JSON nếu chưa kiểm soát workload; một kiến trúc nhiều node không tự có độ trễ thấp hơn nếu phải trả thêm chi phí phối hợp. Các phương án chỉ có thể được xếp hạng sau khi xác định metric, điều kiện và phạm vi bảo đảm cần ưu tiên.

Những công trình trên đã thiết lập nhiều nguyên lý được dùng trong dự án. Khoảng trống của bài tập là cách hiện thực các nguyên lý đó cho một game nhảy tích lực trên browser, với hai implementation simulation, ownership theo phòng và checkpoint gắn vòng đời. Đây là engineering/integration contribution. Research novelty toàn cầu chưa được xác lập. Đóng góp có thể kiểm tra là những bất biến cụ thể và mức cải thiện của baseline có cùng điều kiện, thay vì tuyên bố toàn bộ kiến trúc là mới.

Bảng 2.3: So sánh công trình theo bài toán, đồng bộ, concurrency và giới hạn áp dụng.

Công trình	Bài toán / phương pháp	Đồng bộ / concurrency	Hạn chế và liên hệ dự án
Colyseus [2]	Game nhiều node; discovery và prefetch	Primary/replica theo đối tượng; phân tán xử lý	Khác backend đơn của dự án; gợi mở phân vùng ownership.

Công trình	Bài toán / phương pháp	Đồng bộ / concurrency	Hạn chế và liên hệ dự án
Local-lag và Timewarp [3]	Ứng dụng liên tục; cân bằng phản hồi và nhất quán	Trì hoãn cục bộ và sửa diễn biến; không đặc tả cùng worker pool	Không đồng nhất với replay riêng local player; cần so sánh giả định thời gian.
Orleans [4]	Dịch vụ có trạng thái; virtual actors	Runtime quản lý actor phân tán	SerialExecutor không có placement/failover tương đương.
SEDA [5]	Dịch vụ chịu tải; queue giữa các stage	Event-driven stages, điều khiển tài nguyên	Dự án có giới hạn tĩnh; chưa chứng minh adaptive overload control.
Claypool [6]	Tác động latency tới hành động	Phân loại độ chính xác/thời hạn; không phải executor game	Không có user study trong dự án; không suy gameplay performance từ RTT.
Gambetta [7]	Phản hồi trước round trip	Sequence, prediction và reconciliation	Nguồn thực hành; dự án phải bổ sung generation và cancel.
Fiedler [8]	Bước mô phỏng và chạy bù	Fixed timestep; không đặc tả persistence	Dự án phải kiểm chứng nhịp thực và nợ tick.
InnoDB [9]	Cập nhật dữ liệu cạnh tranh	Locking reads và transaction	Không tự cung cấp idempotency nghiệp vụ; cần revision.

Bảng 2.3 liên hệ từng công trình với quyết định thiết kế và giả định cần kiểm tra trong JumpKing. Các số đo của công trình khác không được chuyển trực tiếp thành kỳ vọng cho dự án vì game, phần cứng và mô hình mạng khác nhau. Tương tự, một giới hạn của hệ thống tham chiếu không tự chứng minh phương án hiện tại tốt hơn nếu cả hai chưa được đặt trong cùng điều kiện.

Kết quả tổng quan vì thế là một bản đồ lựa chọn, không phải bảng xếp hạng công nghệ. Dự án chọn sự kết hợp đủ rõ để kiểm chứng trong phạm vi một backend: quyền quyết định tập trung, phản hồi dự đoán, ownership theo phòng và ghi dữ liệu có phiên bản. Những phần chưa được cung cấp, như chuyển chủ liên máy hoặc recovery toàn bộ Room sau restart, trở thành ranh giới kiến trúc được xác định tường minh.

Chương 3

Phát biểu bài toán

3.1 Yêu cầu và phạm vi phục vụ

Người chơi cần thao tác dễ dự đoán, khôi phục đúng lượt và hiểu trạng thái lưu. Host cần quyền cấu hình và điều khiển vòng đời trận, nhưng quyền này phải được server kiểm tra. Người vận hành cần cấu hình triển khai, giới hạn tài nguyên và khả năng phân biệt lỗi mạng với quá tải. Người đánh giá cần thấy rõ protocol, concurrency, đóng góp kỹ thuật và cách tái lập kết quả. Các nhu cầu đó tạo thành yêu cầu kỹ thuật thay vì chỉ danh sách màn hình.

Input gameplay gồm trạng thái phím, định danh lượt, generation và số thứ tự. Output gồm snapshot có thẩm quyền, sự kiện vòng đời, kết quả và trạng thái persistence. Client không được quyết định vị trí chính thức, thời gian hoàn thành hoặc quyền sở hữu dữ liệu bằng payload tự khai.

Mạng có thể gây trễ, dồn message, đứt kết nối hoặc làm phản hồi của một lệnh không được quan sát. Server có thể hoạt động trong lúc client offline; database có thể tạm không ghi được. Thiết kế không giả định browser, server và database cùng một clock hoặc một miền thất bại.

Nhịp danh nghĩa là 60 tick/giây và 20 snapshot/giây. Hàng đợi và tài nguyên kết nối phải hữu hạn. Lỗi input phải bị cô lập, không làm dừng các phòng khác. Cơ chế phục hồi phải phân biệt reconnect trong thời hạn với membership đã kết thúc. Chất lượng Internet và sức chứa tối đa là thuộc tính cần đo thực tế, không thể suy trực tiếp từ các hằng số cấu hình.

Nhịp 60 Hz và snapshot 20 Hz là mục tiêu vận hành danh nghĩa. Chúng tạo ngân sách thời gian và tần suất truyền dự kiến, nhưng không phải cam kết hard real-time. Tính đúng còn bao gồm hành vi khi không đạt mục tiêu: giới hạn queue, cancel input, timeout, trạng thái lưu chưa xác nhận và luồng khôi phục có định danh.

Phạm vi nghiệm thu được tổ chức theo ba mức: luật chuyển trạng thái; tích hợp HTTP/WebSocket/database; vận hành trên thiết bị và mạng thực. Các mức bổ sung cho nhau. Kết quả trong mô hình cho phép giải thích cơ chế, còn số đo loopback mô tả một môi trường cụ thể. Khả năng vận hành Internet dài hạn cần nhóm đo riêng với thời lượng và miền lỗi phù hợp.

Độ trễ thấp, sử dụng ít tài nguyên và lưu mọi thay đổi tức thời không thể được tối ưu độc lập. Tăng tần suất snapshot có thể giảm độ cũ của hình ảnh nhưng tăng serialization và outbound. Tăng tần suất checkpoint có thể giảm khoảng state chưa lưu nhưng tăng tải giao dịch. Giữ lịch

sử input dài hơn cho phép replay nhiều hơn, đồng thời làm tăng chi phí correction và mức state cũ được tích lũy.

Định hướng của dự án là bảo toàn luật và quyền quyết định trước, sau đó lựa chọn mức phản hồi và chi phí trong phạm vi có thể vận hành. Tính đúng của dữ liệu lưu được thể hiện bằng commit và trạng thái xác nhận, không bằng giả định mọi thao tác đều được ghi ngay. Thứ tự ưu tiên này làm cơ sở cho việc đánh giá phương án ở các chương thiết kế và thực nghiệm.

3.2 Mô hình trạng thái, thành phần và định danh

Xét tập client $C = \{c_1, \ldots, c_n\}$ giao tiếp với một backend B qua HTTP và WebSocket. Backend sở hữu tập phòng R và gửi yêu cầu ghi tới database D. Mô hình không giả định đồng hồ các máy đồng bộ. Nó cũng không đồng nhất sự tồn tại của kết nối với quyền tham gia một lượt chơi: quyền đó được xác định bởi session, membership và ngữ cảnh game.

Ba miền lỗi chính là client, backend và database. Client có thể bị đình trệ hoặc mất mạng trong khi backend tiếp tục hoạt động. Backend có thể dừng trong khi dữ liệu đã commit vẫn còn ở database. Database có thể từ chối một lần ghi khi simulation vẫn giữ state trong RAM. Sự phân tách này quyết định phạm vi phục hồi: nối lại socket không thể tự khôi phục Room đã mất khỏi RAM; đọc checkpoint không thể khôi phục phần tiến độ chưa commit.

State mô phỏng tại tick k được mô hình hóa bởi $S_k = (P_k, A_k, W_k)$, lần lượt là physics, adventure và world. State kết nối và membership được ký hiệu M, còn bản dữ liệu bền vững là D_r tại revision r. Renderer sử dụng một phép chiếu từ state để tạo hình ảnh; persistence sử dụng một biểu diễn phục vụ phục hồi hoặc thống kê. Các biểu diễn này có thể chia sẻ thông tin nhưng có mục đích và vòng đời khác nhau.

$$V_k = \pi_{\text{render}}(S_k), \qquad D_r = \pi_{\text{persist}}(S_{k_r}, M). \tag{3.1}$$

Trong biểu thức (3.1), k_r là tick gắn với payload đã ghi, không nhất thiết bằng tick đang hiển thị. Công thức là mô hình phân tích của báo cáo, không phải khẳng định implementation lưu toàn bộ M. Trên thực tế, Room không lưu dài hạn toàn bộ state vật lý hoặc reconnect token. Phép chiếu nhấn mạnh rằng cần chỉ rõ phần nào được lưu trước khi suy luận khả năng phục hồi.

Account xác định chủ thể nghiệp vụ; session xác định phiên xác thực. Membership xác định vị trí tham gia trong sảnh/phòng, còn playId xác định lượt hoặc trận. Generation tách những lần thiết lập lại ngữ cảnh của cùng lượt. Sequence sắp thứ tự input trong ngữ cảnh; requestId liên hệ control request với phản hồi; revision điều phối thứ tự ghi dữ liệu.

Nếu bỏ generation và chỉ so sequence, reconnect có thể tạo ra hai nguồn input cùng khai những giá trị tương tự. Nếu dùng requestId thay revision, một request mới vẫn có thể mang state cũ tới database. Nếu dùng room code thay quyền owner, người biết mã phòng có thể bị nhầm thành người sở hữu một checkpoint. Những trường hợp này cho thấy định danh có ý nghĩa theo phạm vi, không chỉ là các chuỗi phục vụ truy vấn.

3.3 Mô hình lỗi và điều kiện khôi phục

Chậm và mất liên lạc có thể có cùng biểu hiện trong một khoảng quan sát hữu hạn. Nếu chưa có phản hồi, client không biết server chưa xử lý, đã xử lý nhưng phản hồi chưa tới hay đã dừng. Timeout vì vậy là quyết định của ứng dụng dựa trên thời hạn quan sát, không phải phép chứng minh nguyên nhân vật lý. Hệ thống cần hành vi an toàn sau quyết định đó và một cách truy vấn state khi liên lạc phục hồi.

Ngữ cảnh đã kết thúc lại là một tình trạng nghiệp vụ: lượt có thể đã terminal, membership hết hạn hoặc người chơi đã Leave. Nối lại transport không có quyền làm ngữ cảnh đó sống lại. Phân loại ở Bảng 3.1 giúp UI đưa ra lựa chọn phù hợp: khôi phục membership còn hợp lệ khác với bắt đầu lượt mới; checkpoint đã lưu khác với state Room chỉ tồn tại trong process.

Bảng 3.1: Phân biệt các tình trạng thường cùng xuất hiện dưới thông báo mất kết nối.

Tình trạng	Dữ kiện có thể quan sát	Hệ quả thiết kế
Phản hồi chậm	Request hoặc ACK chưa hoàn tất	Giới hạn pending; không tự lặp mọi mutation.
Transport đóng	Socket không còn khả dụng	Dừng input; thực hiện recovery có quyền và context.
Input tồn đọng	Tuổi queue hoặc pending tăng	Phân biệt truyền chậm với consume chậm.
Membership kết thúc	Server trả trạng thái hết hạn/không còn tham gia	Kết thúc luồng Resume/Reconnect cũ.
Backend restart	State Room RAM mất, dữ liệu commit còn	Phục hồi theo chính sách của từng loại dữ liệu.

Tính an toàn trả lời điều không được xảy ra, như áp input sai generation hoặc ghi lùi revision. Tính tiến triển trả lời thao tác có thể đi tới kết quả nào, như request được phản hồi hoặc người chơi có đường thoát khỏi màn hình chờ. Hai thuộc tính có thể tạo đánh đổi: từ chối một input quá cũ bảo vệ state nhưng làm gián đoạn điều khiển; chờ database vô hạn giữ hy vọng lưu nhưng làm thao tác không kết thúc.

Trong mô hình hiện tại, các queue và deadline tạo biên hữu hạn cho tài nguyên và thời gian chờ theo chính sách. Chúng không bảo đảm mọi request hợp lệ luôn thành công dưới tải tùy ý. Thay vào đó, hệ thống cần một kết quả lỗi hoặc trạng thái chưa xác nhận có thể hiểu được. Đối với kiểm thử, chỉ xác nhận không bị crash chưa đủ; cần quan sát cả việc pending được giải phóng và UI còn đường thực hiện hành động tiếp theo.

3.4 Các bất biến và tiêu chí kiểm chứng

1. Kết quả chính thức xuất phát từ simulation của server.
2. ACK không vượt quá input thực sự đã được consume.
3. Input thuộc lượt/generation cũ không ảnh hưởng ngữ cảnh hiện tại.
4. Retry không tạo thêm kết quả nghiệp vụ cho cùng định danh.

5. Revision cũ không ghi đè trạng thái mới; lượt terminal không bị autosave phục hồi.
6. Thông báo SAVED chỉ xuất hiện sau khi ghi dữ liệu thành công.
7. Client chậm không khiến vòng mô phỏng phải chờ socket của client đó.

Các bất biến được chuyển thành tình huống kiểm tra ở Bảng 3.2; đây là tiêu chí thiết kế test, không phải chứng minh hình thức cho mọi interleaving.

Bảng 3.2: Từ bất biến đến tình huống kiểm tra.

Bất biến	Điều kiện cần bảo toàn	Tình huống có thể bác bỏ
ACK đúng	Sequence được ACK đã được consume	Chỉ enqueue mà ACK đã tăng.
Ngữ cảnh đúng	Input chỉ tác động đúng playId/generation	Socket cũ gửi input sau reconnect.
Quyền đúng	Thao tác dựa trên owner/Host hiện tại	Client tự khai playerId của người khác.
Revision đúng	Writer cũ không ghi đè bản mới	Autosave cũ tới sau Finish.
Lưu đúng	Xác nhận lưu tương ứng commit	UI báo Saved dù transaction rollback.
Restore đúng	State lỗi không để lại cập nhật một phần	Physics được đổi trước khi adventure bị từ chối.

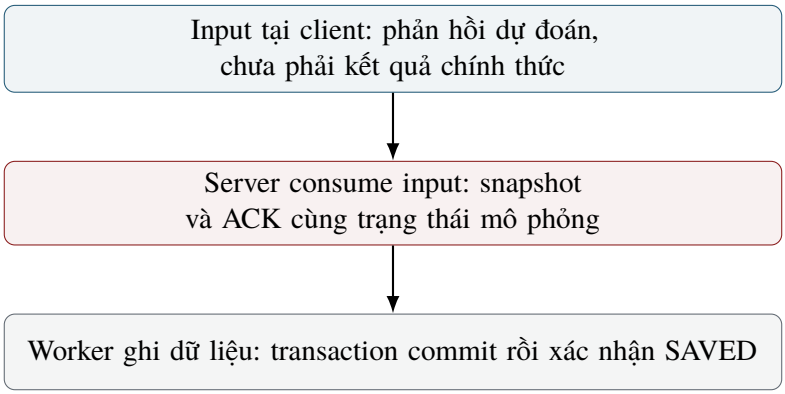
Một test tốt cần làm cho phản ví dụ có khả năng xuất hiện, rồi xác nhận kết quả kỳ vọng bằng dữ liệu quan sát được. Chẳng hạn, test restore chỉ kiểm tra không có exception sẽ không nhận diện cập nhật một phần; nó cần so sánh state trước và sau lần restore bị từ chối. Tương tự, test retry cần kiểm tra số hàng và nội dung kết quả, thay vì chỉ đếm số phản hồi thành công.

Chương 4

Phương pháp, kiến trúc và giao thức

4.1 Nguyên lý và kiến trúc đồng bộ

Thiết kế tách ba mốc ở Hình 4.1: client dự đoán hành động, server consume input và database commit kết quả. Mỗi mốc giải quyết một nhu cầu riêng. Client prediction ưu tiên tính đáp ứng; server simulation ưu tiên quyền quyết định; transaction ưu tiên độ bền và tính nhất quán. Việc phân tách giúp tránh một lỗi nhận thức phổ biến: xem hình ảnh đã xuất hiện là bằng chứng dữ liệu đã được xác nhận lâu dài.



Hình 4.1: Ba mốc xác nhận và ba ý nghĩa khác nhau của trạng thái.

Gọi S_k là trạng thái mô phỏng tại tick k, I_k là input được áp dụng và $\Delta t = 1/60$ giây. Mô phỏng được mô tả bởi

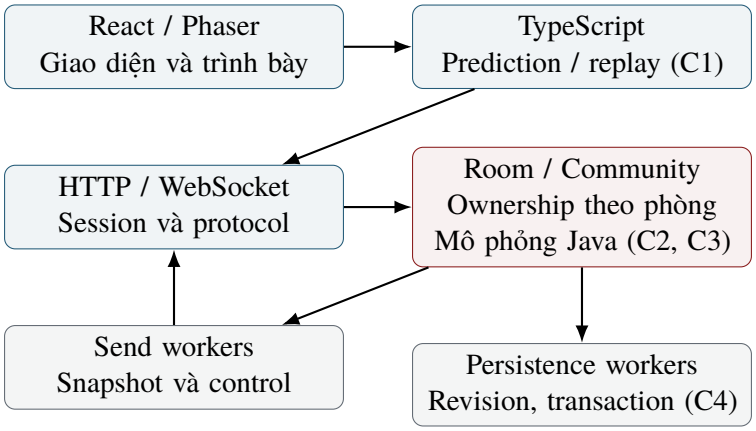
$$S_{k+1} = F(S_k, I_k, \Delta t). \tag{4.1}$$

S_k bao gồm nhiều hơn tọa độ: vận tốc, charge, buffers, trạng thái adventure và world đều có thể ảnh hưởng tương lai. Vì vậy, phục hồi chỉ vị trí sẽ không đủ để replay đúng. Thời gian UTC phục vụ timestamps và thời hạn phiên; monotonic clock phục vụ lịch chạy; tick mô phỏng phục vụ gameplay. Không được thay thế các đại lượng này một cách tùy tiện.

Simulation không phụ thuộc framework giao diện hoặc database. State được sở hữu bởi executor của phòng; công việc ngoài phòng nhận payload bất biến. Hệ thống ưu tiên phát hiện mất đồng bộ thay vì âm thầm ACK input đã bỏ. Đối với lưu dữ liệu, lỗi phải biểu hiện thành

trạng thái chưa xác nhận hoặc thất bại, không chuyển thành thành công giả để làm trải nghiệm có vẻ liên tục.

Trong kiến trúc ở Hình 4.2, React quản lý màn hình, điều hướng và giao diện. Phaser hiển thị sprite, camera và animation; physics chính thức không do Phaser quyết định. TypeScript simulation cung cấp prediction; network adapter quản lý HTTP/WebSocket và ngữ cảnh kết nối. Spring Boot cung cấp API, authentication, transport và điều phối phòng; MySQL lưu dữ liệu lâu dài.



Hình 4.2: Kiến trúc logic và vị trí các đóng góp C1–C4; mũi tên biểu diễn trách nhiệm trao đổi.

Map, tuning và rules bất biến nằm trong dữ liệu dùng chung; state mỗi lượt nằm riêng. Cách tách này tránh sao chép định nghĩa lớn cho từng người chơi, đồng thời ngăn một lượt sửa dữ liệu của lượt khác. Common chứa ID nội dung, geometry và trigger; đường dẫn âm thanh, font và lời thoại thuộc presentation phía client.

API và simulation không trả managed Entity trực tiếp. Khi cần gửi hoặc lưu, hệ thống tạo biểu diễn phù hợp với quyền người nhận và vòng đời I/O. Đây là điều kiện quan trọng để state đang thay đổi không bị worker đọc vào thời điểm không nhất quán.

Production tách frontend Cloudflare Pages và backend HTTPS/WSS trên VPS, MySQL nội bộ. Phương án đóng frontend cùng JAR hỗ trợ cùng origin; development dùng Vite proxy. Một backend sở hữu membership và phòng trong RAM. Vì vậy, kiến trúc hiện tại không có cơ chế chuyển chủ phòng sang instance khác khi backend chết.

4.2 Giao thức và vòng đời thông điệp

HTTP phục vụ xác thực, truy vấn và thao tác tài nguyên; WebSocket phục vụ luồng gameplay hai chiều. Server dùng session cookie, kiểm tra Origin và quyền Account. Kết nối được mở chưa có nghĩa người chơi đã có membership: các lệnh Join hoặc Reconnect vẫn phải được xử lý và xác nhận. Tương tự, socket đóng không tự đồng nghĩa Leave đã thành công.

```
{
  "v": 1,
  "type": "PLAYER_INPUT",
  "payload": {
    "playId": "run-or-match-id",
    "generation": 2,
    "seq": 42,
    "left": false,
    "right": true,
    "jump": true
  }
}
```

Đây là ví dụ cấu trúc; định danh là giá trị minh họa. Các lệnh điều khiển có thể thêm `requestId`; input không dùng request-response cho từng frame. Bảng 4.1 xác định phạm vi của từng định danh.

Bảng 4.1: Các loại định danh không thể thay thế cho nhau.

Trường	Ý nghĩa
Account	Chủ sở hữu được xác thực bằng session
playId	Lượt Community hoặc trận Room đang tham gia
generation	Thế hệ ngữ cảnh sau restore/reconnect/Resume
seq	Thứ tự input trong ngữ cảnh hiện tại
requestId	Liên hệ yêu cầu điều khiển và phản hồi; phát hiện lặp
revision	Thứ tự phiên bản ghi dữ liệu

Client gửi trạng thái điều khiển theo tick. Server kiểm tra ngữ cảnh và sequence, xếp input FIFO rồi consume tối đa một frame mỗi người trong một bước mô phỏng. ACK chỉ tăng khi frame được consume; snapshot chứa state và ACK tương ứng. Thiếu input mới thì giữ input cuối trong giới hạn timeout; quá hạn sẽ cancel thay vì giả lập một thao tác thả Jump.

Điểm này có ý nghĩa riêng với game tích lực: mất dữ liệu không được chuyển thành một cú nhảy ngoài ý muốn. Một chiến lược chỉ giữ input mới nhất có thể làm mất cạnh nhấn/thả; muốn nén hoặc gộp input phải chứng minh ngữ nghĩa này vẫn được bảo toàn.

Duplicate requestId với cùng nội dung có thể nhận kết quả cũ trong phạm vi cơ chế cache; dùng lại ID cho nội dung khác bị từ chối. Khi timeout, client không tự động phát lại mọi lệnh như Start hoặc New Game. Truy vấn state và đối chiếu định danh an toàn hơn việc suy đoán server chưa thực hiện thao tác.

Ngữ nghĩa xử lý yêu cầu được xác lập theo từng phạm vi kết nối và giao dịch: hệ thống kết hợp giới hạn lặp ở connection với idempotency và transaction ở dữ liệu nghiệp vụ. Khi connection thay đổi hoặc process restart, phạm vi cache và phạm vi dữ liệu bền vững phải được phân biệt.

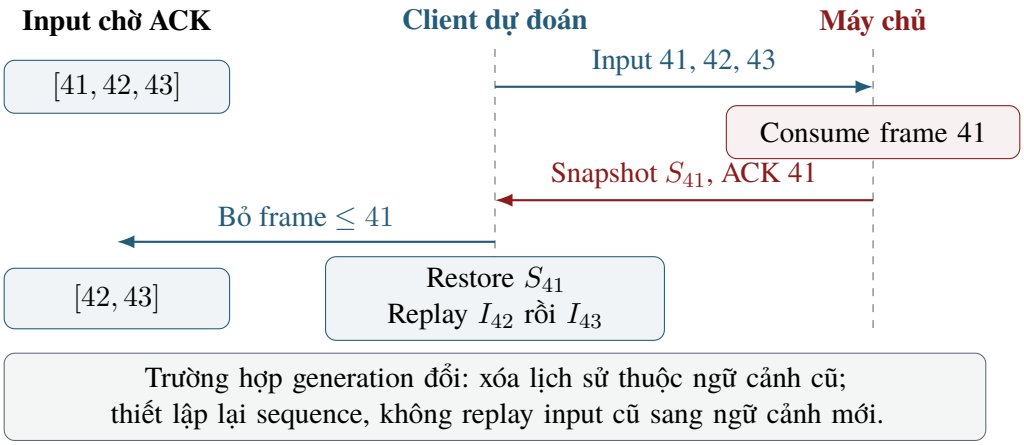
4.3 Thuật toán tiếp nhận và hiệu chỉnh input

Khi người chơi tạo input, client vừa gửi ý định qua mạng vừa đưa input vào simulation cục bộ. Hai đường này có chung nội dung điều khiển nhưng khác thời điểm hoàn tất. Đường cục bộ cung cấp state để renderer phản hồi; đường server tạo state chính thức và ACK. Trạng thái dự đoán vì thế là một giả thuyết thực thi có thể sửa, không phải bản ghi kết quả được tin cậy.

Giả sử client đã tạo các frame 41, 42 và 43 trong một generation. Trong ví dụ ở Hình 4.3, snapshot trả ACK 41 chỉ xác nhận phần lịch sử đến frame đó trong ngữ cảnh hiện tại; client vẫn giữ 42 và 43 để replay. Nếu snapshot có vị trí khác dự đoán, client restore từ snapshot trước rồi mô phỏng lại hai frame còn chờ. Nếu chỉ ghi đè tọa độ mà không phục hồi vận tốc, charge và buffers, việc replay có thể tiếp tục từ một state chưa từng tồn tại trong simulation chính thức.

Sequence được dùng để phát hiện input lùi hoặc trùng. Nó không nhất thiết biểu diễn một luồng số nguyên liên tiếp không có khoảng trống, vì điều kiện server kiểm tra là giá trị mới lớn hơn giá trị đã chấp nhận. Do đó, ACK nên được hiểu theo thứ tự các frame thực sự được nhận và consume, không theo giả định rằng mọi số nhỏ hơn nó đều từng xuất hiện. Điều này đặc biệt quan trọng khi diễn giải test sequence hoặc xây dựng phép gộp input trong tương lai.

Tại server, offer và poll có hai vai trò riêng. Offer kiểm tra context, nội dung và sức chứa; poll xác định frame áp dụng trong bước mô phỏng. Tách hai bước giúp việc xác nhận không bị gắn với thời điểm transport đã parse JSON. Khi queue không có frame mới, simulation có thể tiếp tục với input đang giữ mà ACK không tăng, vì không có input mới nào được consume.



Hình 4.3: Diễn tiến hiệu chỉnh trong cùng một generation và cách loại lịch sử khi đổi ngữ cảnh. Sequence là giá trị minh họa, không phải số đo thực nghiệm; khoảng cách trên hình không biểu diễn độ trễ.

Gọi a là sequence đã được server xử lý và U là tập input chờ ACK. Khi nhận snapshot S, client thực hiện:

1. Xác nhận playId/generation và thứ tự snapshot phù hợp.
2. Loại input có sequence không lớn hơn a.
3. Restore toàn bộ state từ S.
4. Replay phần input còn lại theo đúng thứ tự nếu trạng thái cho phép.

Biểu thức rút gọn là

$$S_{\text{view}} = F^{*}\big(S_{\text{server}}, (I_i)_{i>a}\big). \tag{4.2}$$

F^{*} biểu diễn chuỗi bước mô phỏng, không phải thuật toán khác với luật server. Replay cập nhật state; không được phát lại âm thanh, hội thoại hoặc ending như một sự kiện mới.

Tiếp nhận input. Thuật toán tiếp nhận có đầu vào là envelope và thời điểm monotonic. Đầu ra là một frame được xếp hàng hoặc một lỗi có mã xác định. Kiểm tra context và sequence diễn ra trước khi input ảnh hưởng simulation. Với CANCEL_INPUT, queue được xóa theo ngữ nghĩa hủy; trạng thái neutral không được diễn giải thành một lần thả Jump hợp lệ để launch.

Listing 4.1: Giả mã rút gọn của đường tiếp nhận input.

```
accept(message, now):
    validate_fields_and_types(message)
    require message.context == active_context
    require message.seq > last_accepted
    if message.type == CANCEL_INPUT:
        clear_queued_input()
    require queue_has_capacity()
    enqueue(frame(message), received_at=now)
    last_accepted = message.seq
```

Pseudocode giữ các điều kiện chính của InputFrames; xử lý quyền Fly và các trường tùy chọn được thực hiện trong implementation. Trật tự này cho phép từ chối dữ liệu không hợp lệ mà chưa tạo ra bước vật lý tương ứng. Tính đúng của ACK tiếp tục phụ thuộc vào poll và snapshot, nên chỉ kiểm tra accept chưa đủ bao phủ toàn luồng.

Hiệu chỉnh trạng thái. Đầu vào của reconciliation gồm snapshot, sequence đã xử lý và cờ cho phép replay. Cờ replay phản ánh trạng thái của local player: paused, menu hoặc spectating có thể thay đổi cách sử dụng lịch sử input. Kết quả thuật toán là state dùng cho trình bày tiếp theo và tập frame chưa ACK còn hiệu lực.

Listing 4.2: Giả mã hiệu chỉnh với chuỗi input có thứ tự.

```
reconcile(snapshot, ack, replay_allowed):
    require ack is a valid sequence
    pending = ordered_frames_with_seq_greater_than(ack)
    restore_complete_state(snapshot)
    if replay_allowed:
        for frame in pending:
            if frame.cancel:
                cancel_charge_and_movement()
            simulate_one_tick(frame.input)
```

Phần restore được đặt trước replay để loại tích lũy sai khác từ state dự đoán trước đó. Các sự kiện presentation không đi theo đường replay; nếu âm thanh hoặc ending được phát từ mỗi lần gọi simulation mà không có lớp phân biệt, một correction có thể khiến cùng sự kiện được trình bày nhiều lần. Việc tách state và effect là một yêu cầu của thuật toán, không chỉ là cách tổ chức thư mục.

4.4 Tương đương mô phỏng và trình bày trạng thái

Java chạy luật chính thức; TypeScript chạy dự đoán cục bộ. Hai implementation dùng cùng định nghĩa map/tuning nhưng không dùng cùng mã máy. Chỉ chia sẻ JSON không tự bảo đảm kết quả giống nhau: thứ tự collision, xử lý input, cập nhật charge và điểm ép float32 đều quan trọng. Fixtures và replay giúp phát hiện sai khác ở các trạng thái có ý nghĩa.

Mô hình này có lợi thế là server không phụ thuộc renderer và client không phải chờ từng kết quả mạng. Đánh đổi là chi phí duy trì hai implementation. Một thay đổi physics phải cập nhật hai phía và kiểm chứng đồng thời, tránh tình huống prediction liên tục bị sửa do khác luật.

Một bộ tuning chung giúp hai implementation sử dụng cùng tham số, nhưng sự tương đương còn phụ thuộc thứ tự phép toán. Java float và phép ép Math.fround trong TypeScript tạo những điểm làm tròn có thể kiểm tra. Nếu một phía làm tròn trước collision trong khi phía còn lại làm tròn sau cập nhật vị trí, trạng thái tại biên platform có thể khác nhau dù công thức đại số nhìn giống nhau.

Một thay đổi physics vì thế cần được đánh giá theo chuỗi tác động: tham số hoặc phép tính thay đổi, state biên nào bị ảnh hưởng, expected fixture nào cần kiểm tra và replay dài có còn đi qua các mốc đã biết hay không. Việc chỉ so vị trí sau một tick khó phát hiện lỗi tích lũy. Ngược lại, replay dài chỉ báo sai ở cuối có thể khó định vị nguyên nhân. Hai mức kiểm tra bổ sung cho nhau bằng khả năng phát hiện và khả năng chẩn đoán.

Checkpoint phải chứa các thành phần quyết định diễn biến tương lai: physics, adventure và world. Restore lỗi phải bị từ chối thay vì âm thầm thay tiến độ bằng spawn. Việc kiểm tra state trước khi thay live state làm giảm nguy cơ một lần restore không hợp lệ để lại đối tượng chỉ được cập nhật một phần.

Peers được hiển thị qua bộ đệm snapshot với độ trễ mặc định hiện tại 50 ms. Hệ thống không extrapolate vô hạn khi thiếu dữ liệu. Screen, positionRevision, falls hoặc việc tick lùi giúp nhận diện gián đoạn cần snap thay vì kéo nhân vật qua không gian không hợp lệ. Độ trễ interpolation là đánh đổi mượt hình lấy độ trễ trình bày, không phải tối ưu miễn phí.

Interpolation sử dụng các snapshot như mẫu của một diễn biến đã xảy ra. Độ trễ bộ đệm tạo khả năng có hai mẫu bao quanh điểm hiển thị, nhưng cũng làm peers được nhìn thấy chậm hơn. Khi thiếu mẫu mới, tiếp tục ngoại suy có thể khiến nhân vật vượt quá vị trí được server xác nhận. Dự án giới hạn điểm hiển thị theo mẫu hiện có và xử lý các dấu hiệu gián đoạn riêng.

Respawn và chuyển screen không phải chuyển động liên tục để nội suy. Việc kiểm tra positionRevision, screen, falls hoặc tick lùi giúp renderer nhận diện sự thay đổi loại này. Nếu mẫu cũ vẫn tham gia phép nội suy sau respawn, nhân vật có thể bị kéo về state trước đó dù mạng đang hoạt động bình thường. Như vậy, chất lượng hiển thị phụ thuộc cả thông tin ngữ nghĩa trong snapshot, không chỉ thời gian nhận packet.

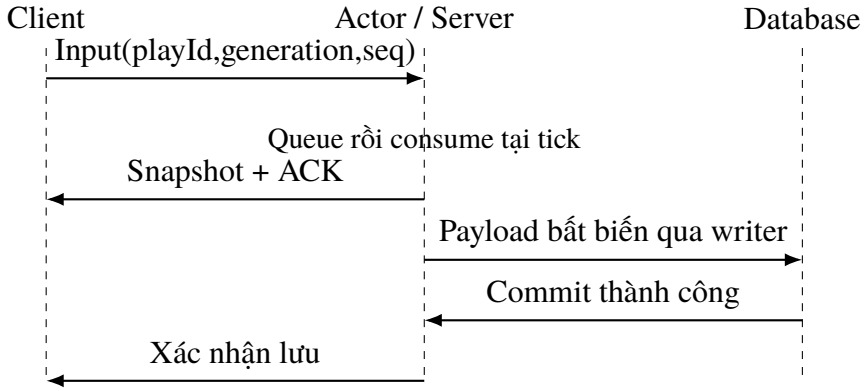
4.5 Đánh đổi thiết kế và trình tự xác nhận

Bảng 4.2 đối chiếu ngữ nghĩa và chi phí dự kiến, chưa phải benchmark. Delta snapshot cần quản lý baseline và xử lý khi baseline mất hiệu lực; snapshot đầy đủ của owner ưu tiên restore chính xác, đánh đổi kích thước truyền.

Bảng 4.2: Các phương án thiết kế được đối chiếu theo ngữ nghĩa và chi phí.

Quyết định	Lựa chọn hiện tại	Đánh đổi và phương án thay thế
Input FIFO	Giữ thứ tự các cạnh điều khiển	Latest-only giảm backlog nhưng có thể mất nhấn/thả; cần thiết kế nén có ngữ nghĩa.
ACK sau consume	Gắn với state đã mô phỏng	ACK khi nhận giảm lượng lịch sử client giữ nhưng không xác nhận thực thi.
Snapshot đầy đủ cho owner	Đủ restore/replay	Delta giảm payload nhưng cần baseline và xử lý mất hiệu lực baseline.
JSON envelope	Dễ kiểm tra schema và debug	Binary có thể giảm byte nhưng cần đối chứng chi phí và khả năng quan sát.
Generation	Tách các lần thiết lập ngữ cảnh	Chỉ reset sequence không đủ phân biệt socket/context cũ.
Cancel tường minh	Hủy charge khi cần dừng điều khiển	Suy từ thiếu frame có thể thay đổi ý nghĩa Jump.

Hình 4.4 thể hiện hai đường xác nhận: input được tiêu thụ tại simulation và payload được commit qua writer.



Hình 4.4: Luồng tổng hợp: ACK gameplay và xác nhận commit có ngữ nghĩa khác nhau.

Gọi λ là tốc độ input đến và μ là tốc độ consume thực tế. Xấp xỉ trong khoảng quan sát có backlog, $\Delta Q/\Delta t \approx \lambda - \mu$. Đây là quan hệ kế toán hàng đợi, không phải mô hình xác suất đã được fit bằng dữ liệu. Hai đầu cùng cấu hình 60 Hz không bảo đảm $\lambda = \mu$ tại mọi thời điểm; scheduling, browser stall và các lượt chạy bù tạo biến thiên. Vì thế cần quan sát tuổi input và ACK, thay vì suy ổn định từ tick rate danh nghĩa.

Buffer hữu hạn chặn mức tích lũy tài nguyên; khi tốc độ đến duy trì vượt tốc độ consume, hệ thống vẫn phải từ chối hoặc tái lập ngữ cảnh. Chính sách phục hồi đánh đổi tính liên tục với yêu cầu không áp input quá cũ. Thứ tự trong executor và transaction chỉ có hiệu lực trong phạm vi tương ứng, không tạo bảo đảm exactly-once cho toàn hệ thống.

Chương 5

Hiện thực và độ tin cậy

5.1 Ownership và chính sách thực thi theo phòng

Mỗi Room hoặc sảnh Community được xử lý tuần tự bởi executor sở hữu nó. Các phòng có thể chạy đồng thời trên worker pool. Đây là tổ chức theo phong cách actor; không đồng nghĩa một phòng có một thread riêng hoặc hệ thống sử dụng một framework actor cụ thể. Lợi ích chính là giảm số vị trí cần khóa state gameplay và làm thứ tự chuyển trạng thái dễ suy luận.

Callback I/O phải quay về executor sở hữu trước khi thay state phòng. Nếu callback database sửa trực tiếp dữ liệu giữa một tick, việc tách I/O ra worker sẽ tạo race condition thay vì giải quyết nó. Do đó, ranh giới ownership là điều kiện cốt lõi, còn worker pool chỉ là phương tiện thực thi.

SerialExecutor bảo vệ thao tác với hai queue bằng đồng bộ hóa, nhưng chạy task ngoài vùng giữ lock. Thiết kế này cho phép task mới được enqueue trong lúc một task đang thực hiện mà không cho hai task cùng drain state của executor. Trường running phối hợp việc lập lịch drain; khi không còn task, executor kết thúc lượt chạy và cho phép một lượt drain mới được lập lịch sau đó.

Điểm tuyến tính hóa ở đây là việc task được đưa vào một hàng đợi xác định, không phải toàn bộ nghiệp vụ đã hoàn tất. Thứ tự trong normal queue và completion queue còn chịu chính sách ưu tiên. Vì drain kiểm tra completion trước, callback hoàn tất có thể được chọn trước một command đang chờ trong queue còn lại. Mô hình đúng là một chuỗi thực thi theo chính sách chọn queue, không phải FIFO tuyệt đối của mọi sự kiện tính theo thời gian đến.

Completion đã gắn với công việc hệ thống nhận trước đó, chẳng hạn một lần ghi cần thông báo kết quả. Tách queue giúp input mới không chiếm hết chỗ của các callback đó. Tuy vậy, completion queue vẫn hữu hạn và cần được tính theo số công việc đang bay; việc ưu tiên không tự tạo bảo đảm fairness cho mọi command dưới tải bất kỳ.

Do drain chạy nhiều task cho tới khi queue rỗng, thời gian task và số lượng task ảnh hưởng khả năng chia sẻ worker với phòng khác. Bounded queue giới hạn backlog nhưng không giới hạn trực tiếp thời gian một task đơn lẻ chiếm CPU. Đây là lý do I/O cần được tách khỏi task gameplay, đồng thời các phép đo scheduling delay có giá trị bổ sung cho compute tick.

Bảng 5.1 xét executor đang chạy task A, trong khi command B đã chờ ở normal queue. Trước khi A kết thúc, callback C được đưa vào completion queue. Khi drain lấy task tiếp theo, C được

chọn trước B. Đây là ví dụ phân tích trực tiếp từ thứ tự poll trong implementation, không phải một phép đo thời gian thực thi.

Bảng 5.1: Một interleaving hợp lệ của hai loại queue.

Mốc	Đang thực hiện	Normal queue	Completion queue
t_0	A	Rỗng	Rỗng
t_1	A	B	Rỗng
t_2	A	B	C
t_3	C	B	Rỗng
t_4	B	Rỗng	Rỗng

Kịch bản cho thấy hai thuộc tính có thể cùng đúng: không có hai task của executor sửa state đồng thời, nhưng thứ tự thực hiện không phải thứ tự đến toàn cục của mọi loại sự kiện. Khi giải thích tính đúng của một callback, cần dùng thứ tự được chính sách cho phép. Chẳng hạn, một callback lưu có thể hoàn tất trước command còn đang chờ, nên điều kiện áp dụng vẫn phải kiểm tra đối tượng và revision của thao tác đã tạo callback đó.

Nếu K_n và K_c là sức chứa hai queue, số task đang giữ bên trong executor có thể được mô hình hóa bởi

$$N_{\text{owned}} = Q_n + Q_c + \mathbf{1}_{\text{running}}, \qquad N_{\text{owned}} \le K_n + K_c + 1. \tag{5.1}$$

Chỉ báo trong công thức đếm task đang thực thi, còn cờ running của lớp cũng bao gồm giai đoạn drain đã được lập lịch. Đây là cận đếm đối tượng theo mô hình queue, không phải một cận bộ nhớ byte hoặc thời gian đáp ứng. Payload mỗi task có thể khác nhau; các công việc I/O đang thực hiện bên ngoài cũng không nằm trong đại lượng này. Phân biệt cận cấu trúc với năng lực vận hành giúp diễn giải chính xác tác dụng của bounded queue.

5.2 Ngân sách tick và truyền trạng thái

Server dùng monotonic clock để xác định bước mô phỏng đến hạn. Bản `c274c58` xử lý bù tối đa năm bước mỗi lượt actor và giữ nợ tick cho lượt sau khi còn mô phỏng. Khi dừng mô phỏng hoặc Room không còn kết nối, clock được đặt lại theo điều kiện vòng đời, không mô phỏng bù toàn bộ thời gian nghỉ.

Giới hạn từng lượt tránh một nhiệm vụ chiếm executor quá lâu. Giữ nợ tick giúp tránh mất vĩnh viễn thời gian mô phỏng sau một gián đoạn ngắn. Tuy nhiên, nếu tổng công việc vượt tài nguyên trong thời gian dài, giữ nợ không tạo thêm năng lực xử lý; nợ có thể tiếp tục tăng và chạm giới hạn phục hồi.

Một lần advance xử lý tối đa năm bước mô phỏng theo điều kiện vòng đời. Giới hạn này thu hẹp số bước trong một task tick, nhưng drain của executor còn có thể thực hiện các command hoặc callback khác trước khi trả worker. Do đó, năm bước không đồng nghĩa một hạn mức CPU tuyệt đối cho cả phòng hoặc bảo đảm mọi phòng được lập lịch trong cùng thời gian.

Chi phí một bước cũng phụ thuộc số người và phần state phải cập nhật. Khi đã giữ nợ tick, tốc độ xử lý thực tế cần có khoảng dư để phục hồi sau gián đoạn; nếu tải liên tục vượt năng lực,

nợ không tự biến mất. Phân tích này nối trực tiếp tới các metric nên đo: thời gian task, scheduling delay, độ sâu queue và tuổi input. Compute tick thấp trong một workload nhỏ là tín hiệu hữu ích, còn quyết định tăng số phòng cần dữ liệu bổ sung về toàn đường thực thi.

State được chụp thành payload bất biến. Snapshot chưa gửi có thể được thay bằng bản mới hơn trước khi serialize; control messages vẫn giữ thứ tự. Điều này phù hợp vì nhiều snapshot trung gian đã lỗi thời đối với renderer, nhưng không thể áp dụng cùng cách bỏ tùy tiện cho input nhấn/thả hoặc mốc lưu bắt buộc.

Gửi socket nằm ngoài tick; client chậm được xử lý qua hàng đợi hữu hạn và deadline. Quota giúp giới hạn thiệt hại khi quá tải, nhưng không được gọi là số người chơi đã được nghiệm thu. Ví dụ, số phòng tối đa cấu hình không chứng minh mọi phòng có thể đồng thời đầy người ở cùng chất lượng dịch vụ.

Việc tách gửi mạng khỏi tick tạo ra một khoảng thời gian giữa chụp và serialize. Nếu payload còn trỏ tới state mutable mà simulation tiếp tục thay đổi, các trường được serialize có thể phản ánh nhiều thời điểm khác nhau. Chụp state bất biến giúp bảo toàn quan hệ giữa ACK, physics và thông tin vòng đời của cùng một snapshot.

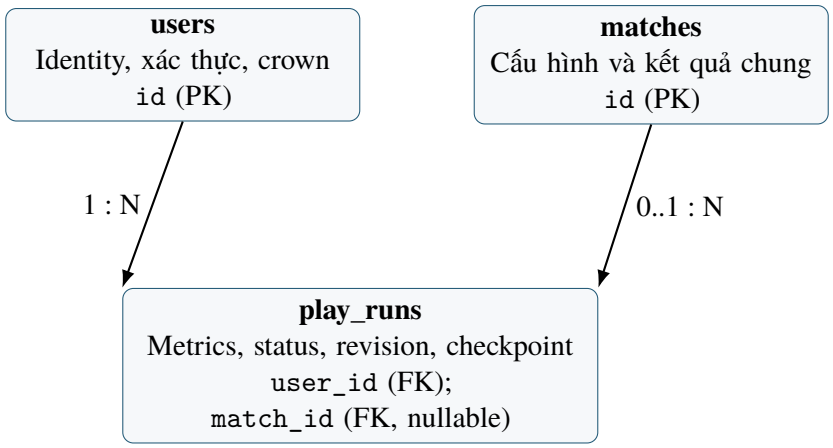
GameConnection giữ snapshot chưa gửi mới nhất, rồi serialize trên send worker. Control messages đi qua queue riêng và được ưu tiên trước snapshot trong drain. Coalescing vì vậy giảm công việc cho các bản state trung gian đã mất giá trị hiển thị; nó không có nghĩa mọi message ứng dụng đều có thể bị thay thế. Các thông báo vòng đời, phản hồi request và mốc ghi có semantics khác snapshot định kỳ, tương ứng các chính sách ở Bảng 5.2.

Bảng 5.2: Chính sách xử lý theo loại dữ liệu bất đồng bộ.

Loại dữ liệu	Chính sách chủ yếu	Lý do ngữ nghĩa
Input	FIFO, consume theo tick	Giữ quan hệ nhấn/thả và charge.
Snapshot định kỳ	Có thể thay bản chưa gửi	State mới có thể thay state hiển thị cũ.
Control reply	Queue giữ thứ tự	Gắn với kết quả yêu cầu và lifecycle.
Autosave	Gộp khi chưa ghi theo chính sách writer	Giảm ghi trạng thái trung gian.
Barrier save	Giữ mốc trong queue	Bảo vệ xác nhận Pause/Leave/Finish.

5.3 Giao dịch và tính nhất quán của persistence

Quan hệ dữ liệu được thể hiện ở Hình 5.1. Database gồm Account, các lượt chơi và thông tin match. Community được phân biệt bởi `match_id IS NULL`; lượt Room liên kết tới match. Status thể hiện vòng đời như ACTIVE, FINISHED, ABANDONED, UNFINISHED và DNF. Snapshot dài hạn chỉ phục vụ Community còn ACTIVE; trạng thái vật lý Room nằm trong RAM.



Hình 5.1: Quan hệ dữ liệu rút gọn từ schema: lượt Community không liên kết match; lượt Room tham chiếu match.

Tổng thống kê được suy ra từ dữ liệu lượt và kết quả đã ghi, tránh duy trì thêm nhiều bộ đếm cùng biểu diễn một sự thật. Leaderboard chọn một best run của mỗi Account trước khi giới hạn số hàng; định danh chỉ ổn định hiển thị, không tùy tiện phá đồng hạng.

Một payload ghi có revision. Revision cũ không được thay thế bản mới. Cùng revision và cùng nội dung có thể là no-op; cùng revision nhưng khác nội dung là xung đột. Trạng thái terminal không được hồi sinh bởi autosave đến muộn. Những điều kiện này giải quyết lỗi thứ tự hoàn tất I/O khác với thứ tự state được tạo.

Giao dịch phối hợp các thao tác cạnh tranh, sử dụng thứ tự khóa thống nhất. Với New Game Community, chốt lượt cũ và mở lượt mới phải nguyên tử: nếu thao tác thất bại, checkpoint cũ không được mất chỉ vì client đã bấm nút. Với Room, finalize phải chốt thông tin trận và roster nhất quán. Khóa và rollback là công cụ bảo vệ các bất biến này, không phải bằng chứng rằng mọi lỗi logic đã được loại bỏ.

Writer nhận payload bất biến, có thể gộp autosave nhưng không bỏ mốc Pause/Leave/Finish. Retry có giới hạn và giữ định danh nghiệp vụ. Trạng thái PENDING, SAVED và FAILED phân biệt việc chấp nhận công việc với kết quả commit. Chỉ sau giao dịch thành công mới có cơ sở thông báo đã lưu.

Nếu database không sẵn sàng, hệ thống có thể tiếp tục mô phỏng trong phạm vi chính sách, nhưng không thể hứa tiến độ mới đã bền vững. Khoảng autosave danh nghĩa không phải cận trên tuyệt đối của dữ liệu có thể mất: retry, hàng đợi và sự cố kéo dài đều có thể làm checkpoint cuối cũ hơn dự kiến.

CheckpointWriter chỉ retry nhóm lỗi dữ liệu tạm thời theo điều kiện implementation; lỗi ràng buộc không được coi là một lần chờ rồi thử vô hạn. Retry giữ payload và revision của job, tránh việc một lần thử sau vô tình chuyển sang state mới mà callback cũ vẫn nhận thành công. Kết quả SUPERSEDED cũng có ý nghĩa khác SAVED: job bị thay thế không có commit riêng tương ứng.

Nếu hàng đợi worker từ chối nhận việc, hệ thống cần kết thúc hoặc báo lỗi các job liên quan thay vì để chúng chờ vô hạn. Phần rejectPending cho thấy việc cleanup không chỉ thu hồi bộ nhớ mà còn phải hoàn tất trạng thái callback. Từ góc độ người chơi, một thao tác kết thúc bằng

FAILED hoặc chưa xác nhận cung cấp thông tin khác một thao tác đã SAVED, dù nhân vật vẫn còn ở vị trí cũ trên màn hình.

Community tiếp tục từ checkpoint đã commit. Room không phục hồi physics sau restart; trận mở được xử lý theo chính sách ABORTED/DNF bằng dữ liệu đã ghi. Session persistence là cơ chế riêng, không phải persistence của trận. Việc giữ được đăng nhập sau restart vì thế không chứng minh có thể tiếp tục trận đang chơi.

5.4 Vòng đời và xử lý tình huống cạnh tranh

Community tự ghép sảnh, mỗi người có tiến độ riêng, Continue và leaderboard. Pause cá nhân phải xử lý trạng thái ổn định và hủy charge phù hợp; một người dừng không phải toàn sảnh dừng. Room có Public/Private, Host, chọn cấu hình, countdown, kết quả và Play Again. Thành viên xác nhận lượt mới riêng, tránh kéo người chưa đồng ý vào trận tiếp theo.

Vòng đời Room ở Hình 5.2 dùng chung cho các chế độ Race, Easy, Hardcore và No Walk; lựa chọn chặng, giới hạn thời gian và Items tùy chọn. Geometry dùng chung, nhưng điều kiện spawn và kết thúc được cấu hình theo chặng. Các chức năng này mở rộng bài toán state machine và snapshot; chúng không tự trở thành novelty nghiên cứu chỉ vì làm sản phẩm nhiều tính năng hơn.

Host có thể tạm dừng Room đang chơi; đồng hồ và physics toàn trận dừng, rồi mốc tick được đặt lại khi tiếp tục. Menu cá nhân không đồng nghĩa Host Pause. Easy nhớ vị trí an toàn riêng cho từng người theo điều kiện luật, giữ trong state trận và snapshot nhưng không tạo checkpoint dài hạn.

Items gồm Banana, Rock và Boots khi Host bật. Server quản lý sở hữu, cooldown, bẫy và tác động; client trình bày hoặc dự đoán trong phạm vi được phép. Những tương tác này làm tăng nhu cầu đối chiếu state và sự kiện. Không được mô tả hệ thống là có collision trực tiếp giữa mọi người chơi khi luật hiện tại không quy định điều đó.

Spectating thay mục tiêu trình bày nhưng không thay owner của ACK, checkpoint hoặc ngữ cảnh input. Ending chỉ phát cho chủ lượt đủ điều kiện trong Community; Room đi tới Results, kể cả full map. Crown là quyền sở hữu dài hạn được cập nhật cùng kết quả hợp lệ; không nhận quyền crown do client khai.

Quạ đen, quạ trắng và các trigger làm phong phú state adventure. Giá trị kỹ thuật liên quan ở đây là state đủ restore, effect không phát trùng khi replay và phân quyền dữ liệu theo người nhận. Tài nguyên nghệ thuật và gameplay lấy cảm hứng từ Jump King phải được phân biệt với phần thiết kế mạng do dự án triển khai.

OnlineRuntime.advance sử dụng performance.now để điều tiết input theo clock thực. Việc tách clock mô phỏng khỏi delta render giúp giới hạn ảnh hưởng của cơ chế làm mượt khung hình đến tốc độ tạo input. Quyền cấp thành tích được kiểm tra tại service và truy vấn dữ liệu: crown cùng Best Finish Time Community chỉ phát sinh từ lượt Community đủ điều kiện. Kết quả Room được quản lý theo match và roster, duy trì ranh giới giữa hai loại tiến trình nghiệp vụ.

Input cũ sau reconnect. Trạng thái ban đầu gồm một membership đang chơi và một socket liên kết với generation hiện tại. Khi reconnect thành công, server thiết lập liên kết mới và ngữ cảnh mới. Một frame cũ có thể vẫn đang chờ xử lý ở tầng giao tiếp; nó không được quyền tác

động simulation chỉ vì Account giống nhau. Kiểm tra playId/generation phân biệt tính hợp lệ của frame với tính hợp lệ của danh tính.

Oracle phù hợp là state và ACK của ngữ cảnh mới không thay đổi do frame cũ. Việc chỉ xác nhận socket mới mở được không bao phủ nguy cơ này. Tình huống cho thấy recovery là một phép chuyển state có điều kiện, không đơn thuần lặp lại handshake. Token và ownership phục vụ quyền khôi phục; generation phục vụ hiệu lực của dòng input sau khôi phục.

Autosave đến muộn. Giả sử payload autosave revision r đã được tạo trước khi lượt kết thúc. Sau đó, payload terminal có revision mới hơn được xử lý. Nếu một write cũ tới muộn, service so sánh revision và status để quyết định chấp nhận, bỏ qua hoặc báo xung đột. Hệ thống không dùng thứ tự hoàn tất thread làm thứ tự nghiệp vụ.

Oracle gồm status terminal, metrics cuối và snapshot sau finalize. Một kiểm tra chỉ đếm số lần gọi writer có thể bỏ sót trường hợp database đã bị ghi lùi. Điều cần bảo toàn là nội dung hàng dữ liệu sau mọi lần retry hợp lệ. Tính chất này được giới hạn trong các điều kiện mà giao dịch và kiểm tra revision thực hiện; nó không được mở rộng thành exactly-once toàn mạng.

Cạnh tranh thay đổi lượt. CommunityRuns.open đối chiếu expectedRunId với lượt ACTIVE quan sát được trong transaction. Khi tạo lượt thay thế, lượt cũ được chốt trước khi insert lượt mới; flush giải phóng ràng buộc ACTIVE theo cách Hibernate sắp thứ tự update/insert. Cùng một yêu cầu từ tab cũ không được tự động thay thế một lượt mới mà tab đó chưa quan sát.

Nếu giao dịch thất bại, rollback phải giữ dữ liệu ở trạng thái nhất quán thay vì để lại thao tác bỏ lượt cũ nhưng chưa có lượt mới. Oracle vì vậy gồm cả identity của run và status/snapshot, không chỉ thông báo trả về client. Đây là tình huống cho thấy optimistic expectation ở lớp ứng dụng và khóa trong database có vai trò bổ sung.

Callback trên ngữ cảnh đã đổi. Writer thực hiện I/O ngoài phòng và báo hoàn tất sau đó. Trong khoảng thời gian này, người chơi có thể Leave, reconnect hoặc đổi lượt. Callback quay về executor để được xử lý tuần tự; tại nơi áp dụng, kết quả cần được liên hệ với đối tượng và thao tác đã tạo nó. Việc chạy callback tuần tự ngăn cập nhật đồng thời, nhưng việc kiểm tra đúng đối tượng vẫn là trách nhiệm nghiệp vụ.

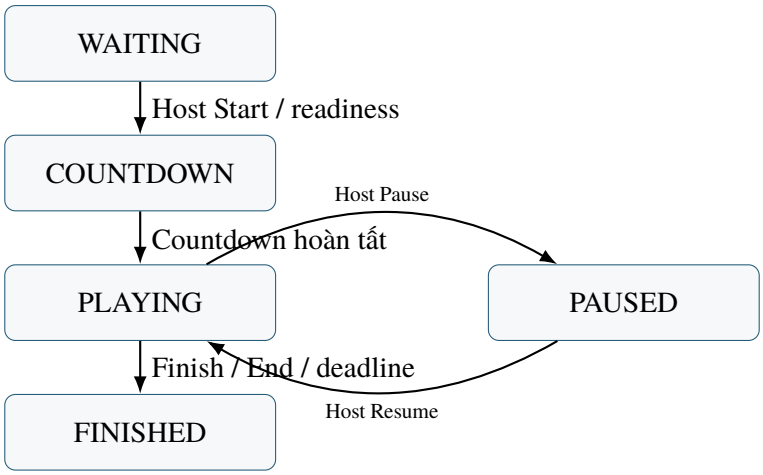
Phân biệt này tránh suy luận quá mức từ mô hình actor-like: tuần tự hóa không sửa được một callback mang sai identity. Test cần xác nhận kết quả cũ không đổi trạng thái lượt mới hoặc phát thông báo lưu cho sai owner. Trong báo cáo, đây là một tiêu chí truy vết và kiểm chứng cần đối chiếu từng đường callback, không phải bảo đảm suy ra từ tên lớp executor.

Hình 5.2 mô tả các trạng thái chính của Room; quyền chuyển trạng thái được kiểm tra tại server.

5.5 Bảo mật và mô hình triển khai

Hệ thống dùng session phía server, password BCrypt và bảo vệ CSRF cho HTTP mutation. WebSocket kiểm tra Account và Origin. Quyền Host, owner và context được server kiểm tra tại thao tác nghiệp vụ. Google Sign-In là phương thức đăng nhập bổ sung; không biến token phía client thành quyền tự ghi kết quả game.

Cookie, CORS và Origin là các cơ chế khác nhau. CORS cho HTTP không thay kiểm tra Origin của WebSocket. Khi frontend/backend khác site, chính sách cookie cần tương thích và



Hình 5.2: Các trạng thái Room chính; reconnect không tự tạo trận mới.

trình duyệt có thể hạn chế third-party cookies. Đưa hai phía về cùng site có thể giảm phụ thuộc này, nhưng không tự giải quyết nghẽn input hoặc CPU.

Session trả lời ai đang xác thực; membership trả lời người đó đang tham gia ở đâu; Host/owner trả lời có quyền thực hiện thao tác nào. Một message hợp lệ về JSON vẫn có thể sai quyền hoặc sai lifecycle. Bởi vậy, validation được phân tầng: kiểm tra cấu trúc ở transport, kiểm tra context ở luồng game và kiểm tra quyền sở hữu dữ liệu ở service.

Giới hạn kích thước, message rate và queue bổ sung kiểm soát tài nguyên. Chúng không thay kiểm tra quyền: một client gửi ít message vẫn có thể gửi thao tác sai owner. Ngược lại, một message đúng quyền nhưng tới với tốc độ quá cao vẫn có thể vượt chính sách phục vụ. Sự tách biệt giúp xác định lỗi được từ chối vì nội dung, quyền, state hay tài nguyên, qua đó hỗ trợ chẩn đoán khi vận hành.

Session cookie, CSRF và kiểm tra Origin giải quyết các ranh giới khác nhau. Tài liệu Spring Security mô tả cơ chế CSRF [10]; OWASP nêu các nhóm kiểm tra WebSocket [11]. Các cơ chế được đối chiếu theo ranh giới tấn công mà chúng bảo vệ; mức an toàn triển khai còn phụ thuộc cấu hình vận hành. Transactionality của JPA cũng cần được đối chiếu với phạm vi transaction ở service thực tế [12].

Frontend trên Pages tải bundle có backend origin gắn lúc build. Backend HTTPS/WSS chạy trên VPS; MySQL không cần được công khai cho trình duyệt. Mẫu cấu hình không chứa bí mật thật. Pipeline giữ JAR trước và restart dịch vụ; upload thành công không thay thế kiểm tra đăng nhập, WebSocket và luồng lưu sau deploy.

Không được hứa zero-downtime với kiến trúc một backend hiện tại. Graceful shutdown giảm rủi ro khi dừng bình thường, nhưng không đồng nghĩa mọi Room đang chạy được chuyển sang process mới. Vận hành phải xem deploy và restart là những nguồn gián đoạn cần thông báo, quan sát và thử nghiệm.

Server-authoritative ngăn việc chấp nhận trực tiếp vị trí/kết quả client tự khai trong protocol hiện tại, nhưng không chứng minh chống mọi gian lận. Session persistence cũng không chống mất file hoặc crash mọi trường hợp. Những phát biểu bảo mật trong báo cáo là về cơ chế và ranh giới, không thay cho penetration test hoặc kiểm toán độc lập.

Chương 6

Thực nghiệm và thảo luận

6.1 Thiết kế thực nghiệm và metric

Thiết kế đánh giá sử dụng ba tập bằng chứng. E_P là mô hình đối chứng snapshot-only và prediction/reconciliation, được chạy lại năm lần với cùng chuỗi input. E_V là tập kiểm thử chức năng ở phiên bản phân tích. E_H là bộ phép đo tải tham chiếu được lưu cùng dự án [13]. Các tập được phân biệt về môi trường, phiên bản và đại lượng quan sát; metadata tái lập được trình bày trong phụ lục.

E_P kiểm tra độ trễ trình bày và sai lệch so với một mô phỏng tham chiếu. E_V kiểm tra các kết quả kỳ vọng của state machine, giao dịch và khôi phục. E_H mô tả tải nhỏ trên loopback, phục vụ phân tích chi phí thay vì ước lượng sức chứa sản xuất. Môi trường của E_H khác môi trường tái kiểm chứng, do đó các giá trị không được gộp thành cùng một mẫu thống kê.

H1: prediction giảm thời gian nhìn thấy hành động so với snapshot-only khi cùng input, map và độ trễ mô hình. H2: trong workload nhỏ được chọn, hệ thống duy trì nhịp snapshot gần 20 Hz và compute tick dưới ngân sách danh nghĩa 16,67 ms. H2 không phải giả thuyết về sức chứa tối đa production.

Baseline snapshot-only hiển thị snapshot nhận được cho nhân vật bản thân, không prediction hoặc interpolation của chính nhân vật. So sánh giữ cùng simulation và input nhằm cô lập tác động của prediction. Không có baseline persistence đồng bộ được đo, nên không được suy ra phần trăm tăng throughput do worker.

Workload live gồm Community 1 người, Room 2 người, Room 10 người và hai Room mỗi phòng 3 người. Sau warm-up một giây, khoảng lấy mẫu khoảng năm giây; input danh nghĩa 60 Hz. CPU gồm cả driver trong JVM; heap là peak lấy mẫu của process và có thể tích lũy qua scenario.

Thử compute tick dùng 600 tick warm-up, 1.800 mẫu cho mỗi roster 2/10 người, không bao gồm scheduler, serialization và I/O. Mô hình prediction chạy 1.800 tick/điều kiện, 15 cú nhảy, trễ 0/50/100 ms mỗi chiều, jitter snapshot 0–2 tick; không packet loss hoặc clock drift.

Đơn vị quan sát của mô hình prediction là một lần hành động trong một chuỗi input xác định; đơn vị của metric sai lệch là state tại tick; đơn vị của workload live là một cấu hình chạy trong cửa sổ lấy mẫu. Các mẫu có thể phụ thuộc nhau vì cùng thuộc một simulation hoặc cùng JVM. Vì vậy, số lượng tick không được diễn giải trực tiếp thành số lần thử độc lập về chất lượng mạng.

Năm lượt tái kiểm chứng có giá trị kiểm tra khả năng lặp lại implementation và pipeline đo. Khi cùng input, cùng lịch trễ mô hình và cùng luật, kết quả trùng nhau là điều phù hợp với thiết kế xác định. Nó không chứng minh một độ biến thiên bằng không trong môi trường Internet. Phân biệt đơn vị quan sát với đơn vị thực nghiệm giúp tránh việc dùng quá nhiều tick để tạo cảm giác chắc chắn thống kê vượt quá dữ liệu.

Đối chứng giữ cùng map, quy tắc charge, nhịp mô phỏng, chuỗi điều khiển và định nghĩa thời điểm quan sát. Biến thay đổi chính là chiến lược phản hồi local: đợi snapshot hoặc dùng prediction/reconciliation. Độ trễ mỗi chiều được khảo sát theo các điều kiện riêng. Bằng cách giữ các yếu tố luật chơi, phép so sánh hướng tới tác động của cơ chế hiển thị thay vì một biến thể game có luật dễ hơn.

Nếu một phép đo thay đồng thời input trace, cấu hình physics và cơ chế đồng bộ, chênh lệch kết quả sẽ có nhiều cách giải thích. Do đó, baseline không chỉ là một con số thấp hơn để so sánh; nó là một implementation tham chiếu được xác định đủ để tái lập điều kiện. Giá trị của mức cải thiện phụ thuộc vào chất lượng thiết kế đối chứng này.

RTT đo từ heartbeat tới PONG trên cùng clock; không phải one-way delay. Snapshot rate là số snapshot mỗi client theo thời gian; tần suất trung bình không thay thế phân bố khoảng cách giữa snapshot. Outbound tính payload ứng dụng, không gồm overhead TCP/TLS. Compute tick là thời gian tính toán phần được đo, không phải độ trễ đầu-cuối. Các chỉ số được sử dụng theo đúng phạm vi đo để phân biệt chi phí tính toán với độ trễ tương tác.

Với tập mẫu độ trễ $L = \{l_1, \ldots, l_n\}$ đã sắp tăng dần, phương pháp nearest-rank chọn vị trí $\lceil pn \rceil$ cho percentile mức p. Trung vị và p95 mô tả những vùng khác nhau của phân bố: trung vị phản ánh trải nghiệm giữa tập mẫu, trong khi p95 nhấn mạnh phần đuôi được quan sát. Khi số mẫu nhỏ, mỗi quan sát có ảnh hưởng lớn hơn tới percentile.

$$\widehat{L}_p = l_{(\lceil pn \rceil)}, \qquad 0 < p \leq 1. \tag{6.1}$$

Biểu thức (6.1) là quy tắc tổng hợp, không tạo ra khoảng tin cậy. Tương tự, trung bình các p95 của nhiều lượt không bằng p95 của tập mẫu gộp nói chung. Vì dữ liệu đã tổng hợp và dữ liệu gốc có độ phân giải khác nhau, mỗi giá trị được diễn giải theo đúng phạm vi metric ở Bảng 6.1.

Bảng 6.1: Metric, phạm vi và diễn giải cần tránh.

Metric	Diễn giải sử dụng	Diễn giải vượt bằng chứng
RTT	Thời gian phản hồi hai chiều theo một clock	Độ trễ một chiều khi chưa đồng bộ clock.
Snapshot/s	Tần suất nhận trung bình theo client	Mọi khoảng cách snapshot đều bằng nhau.
Compute tick	Chi phí tính phần mô phỏng được đo	Chi phí đầu-cuối gồm queue và I/O.
Payload MB/s	Byte ứng dụng quan sát được theo thời gian	Băng thông vật lý gồm toàn bộ TCP/TLS overhead.

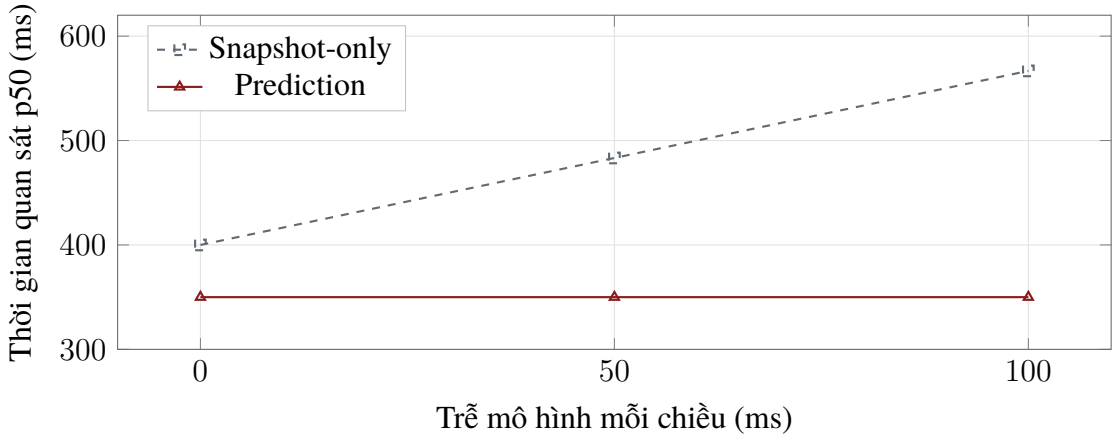
Metric	Diễn giải sử dụng	Diễn giải vượt bằng chứng
CPU core tương đương	CPU time chia wall time của process	Phần trăm CPU riêng của một Room.
Heap peak	Mức heap process ở các lần lấy mẫu	Bộ nhớ riêng từng người chơi.
PASS tests	Oracle đạt trên scenario thực thi	Mọi state hoặc failure đã được bao phủ.

6.2 Kết quả đối chứng prediction

Bảng 6.2: So sánh trong mô hình có kiểm soát [13].

Trễ/chiều (ms)	Snapshot-only p50 (ms)	Prediction p50 (ms)	Sai số p95 baseline (px)	Sai số p95 prediction (px)
0	400,0	350,0	9,09	0
50	483,3	350,0	29,14	0
100	566,7	350,0	42,90	0

Trong Bảng 6.2, hai biến thể cùng tích lực 21 tick, tương ứng 350 ms. Ở điều kiện 100 ms mỗi chiều, mức giảm tuyệt đối là $566{,}7 - 350 = 216{,}7$ ms, khoảng 38,2% so với trung vị baseline. Hình 6.1 thể hiện sự khác biệt giữa hai đường phản hồi. Tỷ lệ trên chỉ mô tả metric và điều kiện trong bảng, không phải mức tăng hiệu năng chung của game.



Hình 6.1: Biểu đồ dựng lại từ bảng lịch sử; không phải phép đo mới.

Năm lượt được chạy lại từ commit c274c58 sau khi đồng bộ dependencies theo lockfile. Mỗi lượt gồm 1.800 tick cho mỗi điều kiện, 15 cú nhảy và cùng chuỗi jitter xác định. Các lượt là kiểm tra tái lập mô hình, không phải năm mẫu mạng Internet độc lập. Các lượt trong Bảng 6.3 cho kết quả giống nhau vì mô hình và input xác định. Miền biến thiên bằng không không phải khoảng tin cậy cho điều kiện mạng thực. Dữ liệu của từng lượt được lưu riêng trong hồ sơ tái thực hiện.

Bảng 6.3: Kết quả mô hình qua năm lượt: trung vị trùng khớp, đơn vị ms.

Trễ mỗi chiều	Snapshot-only p50	Prediction p50
0	400,0	350,0
50	483,3	350,0
100	566,7	350,0

Baseline phải chờ input đi tới server rồi trạng thái quay về theo nhịp snapshot. Prediction cho phép client quan sát kết quả mô phỏng cục bộ ngay theo cùng luật charge. Vì vậy, phần chờ mạng giảm mà thời gian tích lực không đổi. Server vẫn có quyền sửa trạng thái khi nhận input khác với dự đoán.

Sai số bằng không trong điều kiện mô hình không chứng minh game thực tế không correction. Reference là simulation áp input ngay ở client; metric không phải khoảng cách giữa client và server đang có độ trễ ở cùng thời điểm thực. Mô hình cũng không gồm packet loss, clock drift hoặc stall browser, nên kết luận phải giữ trong phạm vi đó.

Thời gian quan sát cú nhảy có hai thành phần về mặt phân tích: thời gian hành động cần theo luật và phần chờ để client thấy kết quả. Trong đối chứng hiện tại, charge là 21 tick, tương ứng 350 ms. Ở baseline, thao tác còn chịu truyền input, xử lý và chờ snapshot quay về. Prediction thay đường hiển thị nhưng không cấp quyền bỏ qua charge hoặc tự xác nhận kết quả.

Với điều kiện trễ 100 ms mỗi chiều, chênh lệch trung vị 216,7 ms lớn hơn một khoảng snapshot danh nghĩa 50 ms vì phần chờ còn chứa đường truyền. Mức giảm tương đối 38,2% sử dụng thời gian baseline làm mẫu số. Đó là cải thiện của metric trình bày trong điều kiện cụ thể, không phải tuyên bố server tính physics nhanh hơn 38,2% hoặc người chơi hoàn thành map nhanh hơn tương ứng.

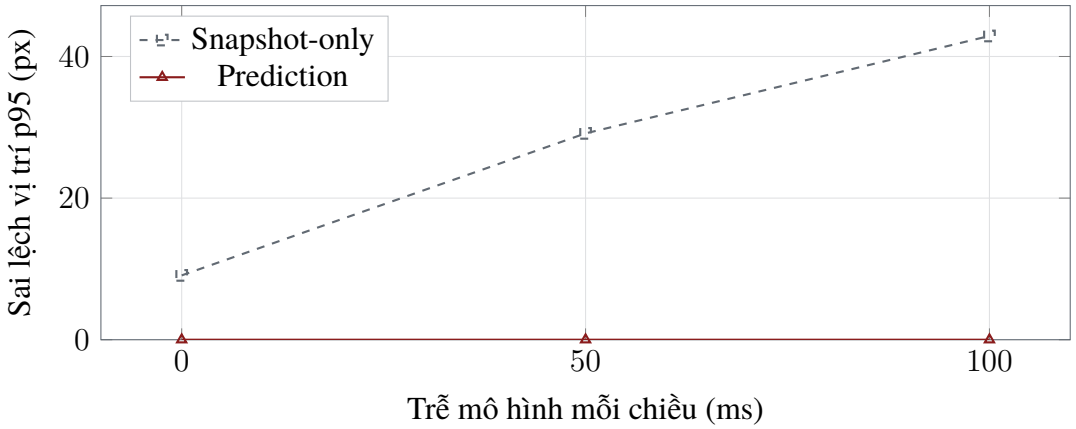
Kết quả ở điều kiện trễ bằng không vẫn có khoảng cách giữa hai biến thể. Điều này phù hợp với việc snapshot-only chịu nhịp cập nhật rời rạc và cơ chế nhận/hiển thị của mô hình. Nó làm rõ rằng latency cảm nhận không chỉ gồm propagation delay: chiến lược đồng bộ và thời điểm công bố state cũng góp phần vào metric.

Bằng chứng hỗ trợ H1 trong các điều kiện mô hình đã chọn: prediction cải thiện thời gian nhìn thấy hành động so với baseline. Chưa có khoảng tin cậy qua nhiều lượt chạy độc lập, vì thế không nên mở rộng thành tuyên bố thống kê về mọi thiết bị hoặc đường truyền. Đánh giá tiếp theo cần thêm độ lớn correction, phân bố ACK lag và phản hồi người chơi, không chỉ latency trung vị.

Hình 6.2 bổ sung góc nhìn về sai lệch trình bày. Giá trị bằng không của prediction chỉ phản ánh điều kiện mô hình xác định, không loại trừ correction khi vận hành qua mạng thực.

6.3 Kiểm chứng chức năng và tính nhất quán

Unit tests kiểm tra các chuyển trạng thái và helper trong phạm vi nhỏ, thuận lợi cho việc định vị sai khác. Integration tests bổ sung sự tham gia của transaction, HTTP/WebSocket và database thật trong các scenario chọn lọc. Test browser kiểm tra đường tương tác và cách giao diện phản ánh state, nhưng có thể chịu ảnh hưởng focus, loading và scheduling của trình duyệt.



Hình 6.2: Sai lệch so với mô phỏng tham chiếu áp input ngay trong mô hình xác định E_P; không phải khoảng cách tới server tại cùng thời gian thực.

Mỗi lớp có oracle phù hợp. Với prediction, oracle là state sau restore/replay. Với writer, oracle gồm số lần ghi và payload; với database, cần kiểm tra hàng dữ liệu, status và revision. Với reconnect, cần đối chiếu membership và ngữ cảnh mới. Một kết quả PASS chỉ có ý nghĩa khi biết test kiểm tra thuộc tính nào và phần nào không nằm trong scenario.

Tập E_V đạt 176 kiểm thử client, 118 kiểm thử đơn vị Java và 16 kiểm thử tích hợp chọn lọc trên MySQL cô lập, không có kiểm thử tích hợp bị bỏ qua trong nhóm được chạy. Nhóm tích hợp gồm TickRecoveryIT, JpaPersistenceIT, CommunityLifecycleIT, RoomRunsIT và RoomRestartIT. Các oracle hỗ trợ phân tích xử lý bù tick, lifecycle, persistence và restart; kết quả không thay thế phép đo performance còn thiếu. Theo bốn RQ, đối chứng mô hình hỗ trợ RQ1; source và kiểm thử hỗ trợ RQ2–RQ3; việc ngoại suy về reliability Internet và sức chứa production trong RQ4 vẫn cần thực nghiệm bổ sung.

6.4 Tải tham chiếu và phạm vi suy luận

Bảng 6.4: Số liệu tải tham chiếu E_H; môi trường loopback [13].

Workload	RTT p95 (ms)	Snapshot/s /client	MB/s outbound	CPU core tương đương	Heap MiB
Community 1	3,44	20,01	0,047	0,66	63,2
Room 2	2,46	20,00	0,109	0,87	79,2
Room 10	2,31	20,06	1,318	1,83	109,5
Hai Room × 3	1,66	20,00	0,382	1,07	144,1

Các cấu hình ở Bảng 6.4 có lần lượt 45, 82, 230 và 186 mẫu RTT; percentile dùng nearest-rank. Mức RTT Room 10 thấp hơn Community 1 không chứng minh tăng người làm mạng nhanh hơn: workload ngắn, số mẫu và điều kiện scheduler khác nhau. Kết luận có căn cứ hơn là các scenario đã quan sát duy trì tần suất trung bình gần nhịp cấu hình.

Outbound tăng mạnh ở Room 10 phù hợp với việc mỗi client nhận trạng thái peers. Không thể ngoại suy tuyến tính vô điều kiện vì số cặp quan hệ người gửi–người nhận, lọc screen, kích

thước snapshot và lịch gửi đều ảnh hưởng tải. Heap cũng không phải bộ nhớ riêng của từng Room để lấy phép trừ giữa hai hàng.

Bảng 6.5: Thời gian compute tick lịch sử, đơn vị ms [13].

Roster	p50	p95	p99	Mẫu vượt 16,67 ms
2 người	0,056	0,087	0,142	0/1.800
10 người	0,188	0,297	0,439	0/1.800

Compute tick ở Bảng 6.5 nằm trong ngân sách của những mẫu này, nhưng chưa bao gồm toàn bộ chi phí để hoàn thành một tick vận hành. Snapshot interval p95 lịch sử nằm khoảng 51,8–67,3 ms dù rate trung bình gần 20 Hz. Quan sát đó minh họa vì sao cần đo scheduler và phân bố thời gian, thay vì chỉ dựa vào chi phí hàm physics.

Harness OnlineWorkloadIT gặp ROOM_NOT_FOUND ở bước Join khi truyền roomId nối với index. Vì scenario thất bại trước khi xuất JSON tổng hợp, không báo throughput, RTT hoặc compute tick mới cho phiên bản này. Failure là kết quả của việc thực thi harness, không đủ để kết luận Join hợp lệ của sản phẩm bị lỗi. Chi tiết từng lần chạy được lưu trong bảng trạng thái và báo cáo chất lượng.

Tính hợp lệ nội tại của đối chứng phụ thuộc việc giữ các biến ngoài chiến lược phản hồi. Tính hợp lệ đo lường phụ thuộc định nghĩa metric và clock. Tính hợp lệ ngoại tại phụ thuộc mức tương đồng giữa môi trường thử với môi trường cần kết luận. Ba vấn đề khác nhau cần được xử lý riêng, vì một phép đo chính xác trong mô hình vẫn có thể không đại diện cho Internet.

Workload tham chiếu có thời lượng ngắn và driver cùng process với server, nên CPU và heap không tách sạch phần sản phẩm khỏi công cụ tải. Lỗi Join của workload hiện hành làm mất khả năng cập nhật bảng tải cho phiên bản phân tích. Phép đo không hoàn tất được loại khỏi tổng hợp hiệu năng, đồng thời số lượt và nguyên nhân được công bố trong hồ sơ bằng chứng. Cách xử lý này bảo toàn ý nghĩa thống kê của bảng kết quả thay vì thay dữ liệu thiếu bằng một giá trị thuận lợi.

6.5 Thảo luận kết quả và ưu tiên cải tiến

Prediction giải quyết thời điểm người chơi thấy phản ứng, server-authoritative giải quyết nguồn quyết định, còn transaction giải quyết sự thật được lưu. Thành công của một lớp không thay cho lớp khác. Một hình ảnh mượt có thể che ACK đang tụt; một kết quả chính thức đúng có thể chưa lưu; một checkpoint hợp lệ có thể không phải tiến độ mới nhất trước sự cố.

Đối với RQ2–RQ3, implementation cung cấp chuỗi cơ chế hợp lý: sequence và generation phân định input; executor phân định ownership; revision và transaction phân định thứ tự ghi. Mức tin cậy tăng khi test tác động trực tiếp vào từng bất biến. Tuy nhiên, chưa có cơ sở gọi chuỗi cơ chế này là một chứng minh toàn diện dưới mọi lỗi phần mềm và phần cứng.

Reconnect có thể xuất phát từ mất kết nối thật hoặc do chương trình chủ động yêu cầu resync. Source mặc định đóng khi input đầu queue chờ ít nhất 500 ms trong khi input mới vẫn đến. Client cũng có giới hạn 180 input chưa ACK, kiểm tra bufferedAmount khoảng 32 KiB và heartbeat có timeout. Những điều kiện khác nhau có thể cùng đưa người chơi tới màn hình Disconnected.

Tuổi queue không phải RTT. Ping thấp không chứng minh game loop không nghẽn; socket còn trả heartbeat không chứng minh mọi input đã consume. Mặt khác, đọc được website HTTP 200 chỉ chứng minh frontend được phục vụ ở thời điểm đó. Không có log và tái hiện phiên chơi, báo cáo không xác định nguyên nhân sự cố cụ thể trên VPS.

Client hiện xử lý onclose bằng thông báo chung, chưa giữ đầy đủ close code/reason/wasClean trong thông báo sự kiện nội bộ. Tối ưu đầu tiên nên là phân loại nguồn đóng, ghi ACK lag, tuổi queue, tick delay và trạng thái tab. Không ghi token hoặc session vào log. Có bằng chứng này mới xác định nên sửa scheduling, giảm chi phí snapshot, điều tiết input hay xử lý đường truyền.

Tăng timeout có thể giảm ngắt nhầm nếu ngưỡng quá chặt, nhưng cũng cho phép state cũ tích lũy lâu hơn. Tăng buffer không giải quyết thiếu năng lực xử lý kéo dài. Resync trên socket còn khỏe là hướng có thể nghiên cứu, với điều kiện generation mới, hủy charge và loại input cũ được xác lập nguyên tử theo ngữ cảnh.

Test phục hồi thành công chứng minh một đường khôi phục, chưa chứng minh sự cố hiếm. Đánh giá độ ổn định kết nối sử dụng tần suất gián đoạn trên giờ chơi và thời gian khôi phục. Validity nội tại bị ảnh hưởng nếu baseline khác cả luật vật lý lẫn cách hiển thị. Validity ngoại tại bị giới hạn bởi loopback, phiên đo ngắn và môi trường đơn máy. Validity đo lường bị ảnh hưởng nếu gọi RTT là one-way delay hoặc gọi compute tick là latency đầu-cuối.

Đối chứng prediction hỗ trợ ưu tiên giữ phản hồi local có hiệu chỉnh thay vì chuyển sang snapshot-only. Phân tích ownership hỗ trợ duy trì đường thay đổi state có chủ rõ ràng. Tuy nhiên, lựa chọn tối ưu tiếp theo cần dựa vào metric của nút thắt thực tế: tuổi queue, scheduling delay, serialize time hoặc thời gian commit, thay vì chỉ nhìn số người chơi cấu hình tối đa.

Nếu ACK tụt trong khi RTT thấp, ưu tiên chẩn đoán nằm ở đường consume hoặc snapshot, không tự động ở chất lượng đường truyền. Nếu bufferedAmount tăng, cần quan sát đường gửi và tốc độ thoát dữ liệu. Nếu lưu thường xuyên ở PENDING, cần tách queue wait với thời gian giao dịch. Một hệ thống quan sát được các giai đoạn này có cơ sở tối ưu chính xác hơn hệ thống chỉ báo một thông điệp Disconnected chung.

Chương 7

Đóng góp và kết luận

7.1 Đóng góp và bằng chứng

Các đóng góp tập trung vào việc kết hợp mô phỏng đa ngôn ngữ, ngữ nghĩa giao thức và quản lý trạng thái bền vững cho trò chơi nhảy tích lực. Prediction, WebSocket và transaction cung cấp nền tảng kỹ thuật; phần phát triển của dự án nằm ở các ràng buộc ownership, ngữ cảnh điều khiển và thứ tự xác nhận. Bốn nhóm đóng góp ở Bảng 7.1 được đối chiếu với phương án gốc, cơ chế hiện thực và bằng chứng tương ứng.

So với một client chỉ nhận snapshot, dự án thêm simulation TypeScript tương ứng Java, dữ liệu chung và kiểm chứng fixtures/replay. Giá trị là phản hồi cục bộ được xây trên cùng luật với nguồn chính thức, thay vì dựng chuyển động minh họa không có quan hệ với physics. Bằng chứng định lượng prediction hỗ trợ tính hữu ích của thành phần này; fixtures hỗ trợ tính tương đương trong các trường hợp được chọn, không chứng minh mọi state khả dĩ.

So với việc áp trạng thái phím mới nhất hoặc ACK khi vừa nhận message, dự án phân biệt accepted/processed, dùng FIFO và generation, đồng thời cancel charge khi mất input. Đóng góp miền bài toán là tránh biến timeout hoặc replay thành thao tác thả Jump ngoài ý muốn. Bằng chứng nằm ở `InputFrames`, prediction và các test context/lag recovery. Hiệu quả của C2 được đánh giá bằng các bất biến điều khiển và kiểm thử tình huống biên.

So với cách để luồng mạng cùng sửa trực tiếp state hoặc chờ gửi/ghi trong tick, dự án dùng ownership theo executor, snapshot bất biến, send worker và persistence worker. Giữ nợ tick qua các lượt actor là một cải tiến cụ thể của bản hiện tại. Bằng chứng cấu trúc và `TickRecoveryIT` chỉ ra chủ đích bảo toàn bước mô phỏng; số liệu compute lịch sử không được dùng để quy toàn bộ hiệu quả cho cải tiến mới.

So với autosave ghi đè không có phiên bản, dự án phối hợp revision, định danh lượt, mốc ghi bắt buộc và transaction. Giá trị là việc xác nhận Pause/Leave/Finish có quan hệ rõ với dữ liệu đã commit. Không có benchmark writer đồng bộ nên chưa thể tuyên bố giảm latency hoặc tăng throughput một tỷ lệ cụ thể nhờ C4.

Bảng 7.1: Cơ chế, giá trị kỹ thuật và bằng chứng của bốn nhóm đóng góp.

Cơ chế	Giá trị kỹ thuật	Bằng chứng	Phạm vi bảo đảm
C1. Mô phỏng Java/TypeScript, dữ liệu chung	Phản hồi cục bộ theo cùng luật mô phỏng	Fixtures, replay và đối chứng E_P	Tích hợp; chưa chứng minh tương đương mọi state
C2. Sequence, generation; ACK sau consume	Bảo toàn nhấn/thả, loại input sai ngữ cảnh	InputFrames; test cancel và context	Chưa có ablation định lượng riêng
C3. Executor theo phòng, worker I/O	Tuần tự cập nhật; tách chờ socket/DB khỏi tick	SerialExecutor, TickRecoveryIT	Chưa đo sức chứa production
C4. Revision, barrier, transaction	Gắn xác nhận lưu với dữ liệu đã commit	Test writer, rollback và integration	Idempotency trong phạm vi nghiệp vụ

7.2 Trả lời câu hỏi nghiên cứu và ý nghĩa kết quả

Đóng góp được phân tích theo ba cấp: cơ chế đã hiện thực, thuộc tính đã được test và mức cải thiện đã có đối chứng. Cơ chế được xác định bằng source và contract. Thuộc tính cần oracle để phân biệt kết quả kỳ vọng với một lần chạy có vẻ bình thường. Mức cải thiện cần baseline và điều kiện đo tương đương. Một đóng góp có thể có bằng chứng mạnh về tính đúng nhưng chưa có đối chứng hiệu năng riêng.

Theo cách phân loại đó, prediction có kết quả định lượng mô hình; generation và cancel có cơ sở ngữ nghĩa cùng test tình huống; serial executor có phân tích ownership và test xử lý thứ tự; persistence có kiểm tra revision và transaction. Bảng 7.2 tổng hợp câu trả lời bốn RQ từ các bằng chứng này; chúng không được cộng cơ học thành một bảo đảm mạnh hơn phạm vi từng phần. Giá trị nghiên cứu nằm ở việc làm rõ quan hệ giữa các phần và phương pháp kiểm chứng của chúng.

RQ	Kết quả hoặc luận điểm	Căn cứ và phạm vi
RQ1	Giảm thời gian quan sát hành động bằng prediction	Đối chứng mô hình, cùng input/charge; server vẫn xác nhận state.
RQ2	Tuần tự hóa theo phòng xác định ownership	SerialExecutor và test recovery; hiệu năng riêng của lựa chọn chưa có ablation.
RQ3	Tách context và revision để xử lý yêu cầu cũ/lặp	InputFrames, writer và integration giao dịch; không mở rộng thành exactly-once toàn mạng.
RQ4	Đánh đổi phản hồi, state tạm và khả năng phục hồi	Mô hình, tải tham chiếu và phân tích kiến trúc đơn backend.

Bảng 7.2: Tổng hợp câu trả lời theo loại bằng chứng.

Sự kết hợp giữa phân tích code, test có oracle và đối chứng mô hình tạo chuỗi kiểm chứng từ yêu cầu tới kết quả. Metric được gắn với harness, bất biến với tình huống kiểm thử và quyết định cập nhật với lớp sở hữu state. Chuỗi này cho phép xác định một sai khác nằm ở giả định thiết kế, hiện thực hay điều kiện đo, thay vì chỉ dựa vào biểu hiện trên giao diện.

Kết quả tích hợp là đường xử lý có các mốc xác nhận tách biệt: phản hồi dự đoán tại client, tiêu thụ input tại server và commit tại database. Khi một phiên bị gián đoạn, ba mốc có thể dừng ở những vị trí khác nhau. Thiết kế context, ownership và revision giúp xác định trạng thái nào còn hiệu lực và phần nào cần khôi phục. Đây là giá trị kỹ thuật xuyên suốt bốn nhóm đóng góp.

Các cơ chế không chỉ có ý nghĩa với map hoặc nhân vật cụ thể. Phân biệt input được nhận với input được thực thi áp dụng cho nhiều ứng dụng điều khiển qua mạng. Generation là một cách phân định hiệu lực của luồng sự kiện sau khi ngữ cảnh được thiết lập lại. Revision và payload bất biến giúp diễn giải I/O hoàn tất khác thứ tự mà không lấy thứ tự thread làm thứ tự nghiệp vụ.

Tuy vậy, chuyển giao sang ứng dụng khác cần đối chiếu ngữ nghĩa. Một hệ thống collaborative editing không có cùng khái niệm tick vật lý; một game có tương tác va chạm dày đặc có thể khó dự đoán hơn local player hiện tại. Kết quả này vì thế cung cấp mẫu phân tích và cơ chế có thể thích nghi, không phải một thư viện bảo đảm tự động cho mọi hệ thống thời gian thực.

7.3 Giới hạn của nghiên cứu

Một backend là một miền lỗi trung tâm. Room trong RAM không tiếp tục physics sau restart. Community chỉ trở lại checkpoint đã commit; session persistence không thay thế checkpoint. Khi database gặp sự cố lâu, hệ thống không thể đồng thời hứa tiến độ luôn bền vững và không bao giờ phải chờ hoặc hạn chế thao tác.

Tài nguyên nội dung Main Babe có attribution trong repository nhưng không được mặc định là tài sản nguồn mở của nhóm. Giấy phép phân phối cần được xác minh theo nguồn quyền sở hữu. Báo cáo phân biệt giá trị kỹ thuật triển khai với quyền sử dụng hình ảnh, âm thanh và nội dung gốc.

Đợt đo lịch sử ngắn, loopback, không có khoảng tin cậy hoặc soak test. Mô hình prediction không phản ánh đầy đủ mạng Internet. Commit hiện tại có nhiều chức năng và tối ưu khác bản đo, nên chưa có cơ sở khẳng định các giá trị lịch sử còn giữ nguyên. Chưa có phép đo tỷ lệ gián đoạn kết nối trong các phiên Internet kéo dài.

7.4 Hướng phát triển

Giai đoạn tiếp theo nên ưu tiên hoàn thiện công cụ đo trước khi mở rộng sản phẩm. Harness tải cần tạo được cùng workload hợp lệ và lưu output theo phiên bản. Sau đó, quan sát được tách theo queue wait, thời gian xử lý, thời gian gửi và commit để xác định nút thắt. Khi đã có đường đo ổn định mới có cơ sở thực hiện ablation hoặc lựa chọn cấu hình tốt hơn.

Mở rộng nhiều backend là bước có phụ thuộc thiết kế lớn hơn. Cần xác định cách định vị chủ phòng, chuyển kết nối tới đúng chủ, xử lý chủ chết và bảo vệ callback hoặc writer cũ khi quyền sở hữu thay đổi. Nếu những vấn đề này chưa được giải quyết, thêm instance có thể tạo nhiều nguồn cập nhật cạnh tranh thay vì tăng reliability. Lộ trình phát triển được xây từ các bất biến cần giữ, không chỉ từ mục tiêu tăng số máy.

Ưu tiên một là quan sát reliability: log nguyên nhân đóng đã chuẩn hóa, ACK lag, tuổi queue, tick scheduling delay và thời gian khôi phục. Ưu tiên hai là lặp workload trên VPS với nhiều client

độc lập, warm-up và khoảng đo dài hơn, công bố phân bố và phiên bản chính xác. Ưu tiên ba là thí nghiệm lỗi có kiểm soát: mất mạng ngắn, DB chậm, restart và tab nền, kiểm tra dữ liệu lẫn trải nghiệm.

Nghiên cứu đối chứng tiếp theo có thể cô lập từng yếu tố trong cùng tải thử, chẳng hạn so sánh cách tổ chức snapshot trước/sau trong harness cô lập. Các cấu hình đối chứng được triển khai trong môi trường thử nghiệm cô lập. Mở rộng nhiều backend chỉ nên thực hiện sau khi thiết kế rõ ownership phòng, routing kết nối, lưu metadata và xử lý chuyển chủ; thêm load balancer đơn thuần không giải quyết các vấn đề đó.

7.5 Kết luận

JumpKing Multiplayer cho thấy bài toán game mạng cần được giải quyết đồng thời ở điều khiển, trạng thái và dữ liệu. Thiết kế kết hợp prediction ở client, mô phỏng có thẩm quyền ở server và giao dịch persistence để phân định các mốc xác nhận. Sequence/generation bảo vệ ngữ cảnh, executor bảo vệ ownership, còn revision và transaction bảo vệ thứ tự dữ liệu.

Đối chứng mô hình và các lượt tái thực hiện hỗ trợ nhận định prediction cải thiện phản hồi trong điều kiện được kiểm soát; workload tham chiếu cho thấy tính khả thi của cấu hình tải nhỏ trên loopback. Các kết quả không xác định sức chứa production và không chứng minh reconnect ngoài thực tế đã được khắc phục. Đóng góp kỹ thuật nằm ở tích hợp mô phỏng đa ngôn ngữ, bảo toàn ngữ nghĩa điều khiển và phối hợp thứ tự xác nhận giữa bộ nhớ với giao dịch.

Các kết quả xác lập cơ sở để phát triển hệ thống theo ba hướng đo được: ổn định ACK và tuổi hàng đợi trong phiên dài, kiểm tra phục hồi dưới lỗi có kiểm soát, và lượng hóa chi phí khi tăng số phòng. Những bước tiếp theo sử dụng cùng các bất biến và mốc xác nhận, nhờ đó việc tối ưu hiệu năng có thể được đánh giá đồng thời với tính đúng của trạng thái.

Tài liệu tham khảo

[1] Ian Fette and Alexey Melnikov. The websocket protocol. Technical Report RFC 6455, RFC Editor, 2011. URL https://www.rfc-editor.org/rfc/rfc6455.

[2] Ashwin Bharambe, Jeffrey Pang, and Srinivasan Seshan. Colyseus: A distributed architecture for online multiplayer games. In *USENIX NSDI*, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/full_papers/bharambe/bharambe_html/.

[3] Martin Mauve, Jörgen Vogel, Volker Hilt, and Wolfgang Effelsberg. Local-lag and timewarp: Providing consistency for replicated continuous applications. *IEEE Transactions on Multimedia*, 2004. URL https://publications.cs.hhu.de/Mauve2004a.html.

[4] Phil Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. Orleans: Distributed virtual actors for programmability and scalability. Technical Report MSR-TR-2014-41, Microsoft Research, 2014. URL https://www.microsoft.com/en-us/research/publication/orleans-distributed-virtual-actors-for-programmability-and-scalability/.

[5] Matt Welsh, David Culler, and Eric Brewer. Seda: An architecture for well-conditioned, scalable internet services. In *SOSP*, pages 230–243, 2001. doi: 10.1145/502059.502057.

[6] Mark Claypool and Kajal Claypool. On latency and player actions in online games. Technical report, Worcester Polytechnic Institute, 2006. URL https://www.cs.wpi.edu/~claypool/papers/precision-deadline/tr-06-13.pdf. Bản technical report trên trang tác giả; không đồng nhất metadata với bản tạp chí.

[7] Gabriel Gambetta. Client-side prediction and server reconciliation, n.d. URL https://www.gabrielgambetta.com/client-side-prediction-server-reconciliation.html.

[8] Glenn Fiedler. Fix your timestep!, n.d. URL https://gafferongames.com/post/fix_your_timestep/.

[9] Oracle. Mysql 8.4 reference manual: Locking reads, n.d. URL https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html.

[10] Spring. Spring security: Cross site request forgery, n.d.. URL https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html.

[11] OWASP. Websocket security cheat sheet, n.d. URL https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html.

[12] Spring. Spring data jpa: Transactionality, n.d.. URL https://docs.spring.io/spring-data/jpa/reference/jpa/transactions.html.

[13] Nhóm dự án JumpKing Multiplayer. Báo cáo thực nghiệm ngày 15/10/2026, 2026. report/report.md, đọc tại commit c274c58; số liệu lịch sử.

[14] Nhóm dự án. Jumking-multiplayer: mã nguồn và tài liệu thiết kế, n.d. URL https://github.com/nvbangg/JumKing-Multiplayer. Phiên bản phân tích: c274c58430fb0f86024b4e485295efab0dfd5656.

Phụ lục A

Điều kiện và khả năng tái lập thực nghiệm

A.1 Phiên bản và môi trường

Phân tích implementation gắn với snapshot c274c58 của repository [14]. Tập E_H là số liệu tham chiếu trong báo cáo thực nghiệm của nhóm [13]; tập E_P là mô hình đối chứng được tái thực hiện năm lượt, còn E_V là nhóm kiểm thử chức năng đã chạy trên môi trường cô lập. Điều kiện quan sát ở Bảng A.1 khác nhau nên các tập không được gộp thành một phân bố.

Bảng A.1: Môi trường của các tập bằng chứng.

Thuộc tính	Tải tham chiếu E_H	Tái kiểm chứng E_P, E_V
Hệ điều hành	Windows 11 Home 10.0.26200	Windows 11 10.0.22621
Bộ xử lý	Ryzen AI 7 H 350; 8 core/16 luồng	Môi trường ghi nhận 12 bộ xử lý logic
Java	25.0.1 theo báo cáo nguồn	JDK 25 được cấp cho runner
Node.js	24.18.0	24.13.1
Database	MySQL 9.6	MySQL 8.4.11, instance test riêng
Mốc chạy	15/10/2026	29/10/2026

Nguồn E_H không gắn đầy đủ SHA của bản đo với từng bảng. Hạn chế này được giữ khi diễn giải dữ liệu tham chiếu. Những mốc thời gian ở đây phục vụ truy xuất điều kiện thực nghiệm, không phải biến giải thích khác biệt về hiệu năng.

A.2 Tham số đo và hồ sơ đối chứng

Mô hình prediction sử dụng 1.800 tick cho mỗi điều kiện, 15 cú nhảy với cùng charge 21 tick, trễ 0/50/100 ms mỗi chiều và chuỗi jitter snapshot 0–2 tick. Năm lượt chạy có cùng trace nên là kiểm tra tái lập mô hình xác định. Workload tham chiếu lấy mẫu khoảng năm giây sau một giây warm-up; compute tick dùng 600 tick warm-up và 1.800 mẫu cho mỗi roster.

Hồ sơ kèm bản nguồn lưu JSON từng lượt, trạng thái kiểm thử, môi trường, SHA-256 và lệnh tái thực hiện. Workload hiện hành chưa tạo kết quả hợp lệ do lỗi Join đã phân tích ở Chương 6; hồ sơ giữ trạng thái thất bại và không điền số đo thay thế.